

Attribute-Based Threshold Issuance Anonymous Counting Tokens and Its Application to Sybil-Resistant Self-Sovereign Identity

Behzad Abdolmaleki¹, Antonis Michalas^{2,5}, Reyhaneh Rabaninejad², Sebastian Ramacher³, and Daniel Slamanig⁴

¹ University of Sheffield, UK

`behzad.abdolmaleki@sheffield.ac.uk`

² Tampere University, Finland

`{reyhaneh.rabbaninejad,antonios.michalas}@tuni.fi`

³ AIT Austrian Institute of Technology, Austria

`sebastian.ramacher@ait.ac.at`

⁴ Research Institute CODE, Universität der Bundeswehr München, Germany

`daniel.slamanig@unibw.de`

⁵ Research Institute of Sweden (RISE), Sweden

Abstract. Self-sovereign identity (SSI) systems empower users to (anonymously) establish and verify their identity when accessing both digital and real-world resources, emerging as a promising privacy-preserving solution for user-centric identity management. Recent work by Maram et al. proposes the privacy-preserving Sybil-resistant decentralized SSI system CanDID (IEEE S&P 2021). While this is an important step, notable shortcomings undermine its efficacy. The two most significant among them being the following: First, unlinkability breaks in the presence of a single malicious issuer. Second, it introduces interactivity, as the users are required to communicate each time with issuers to collect credentials intended for use in interactions with applications. This contradicts the goal of SSI, whose aim is to give users full control over their identities. This paper first introduces the concept of publicly verifiable attribute-based threshold anonymous counting tokens (tACT). Unlike recent approaches confined to centralized settings (Benhamouda et al., ASIACRYPT 2023), tACT operates in a distributed-trust environment. Accompanied by a formal security model and a provably secure instantiation, tACT introduces a novel dimension to token issuance, which, we believe, holds independent interest. Next, the paper leverages the proposed tACT scheme to construct an efficient Sybil-resistant SSI system. This system supports various functionalities, including threshold issuance, unlinkable multi-show selective disclosure, and non-interactive, non-transferable credentials that offer constant-size credentials. The proposed construction is backed by rigorous security definitions and proofs. Finally, our benchmark results show an efficiency improvement in our construction when compared to CanDID all while accommodating a greater number of issuers and additionally reducing to a one-round protocol that can be run in parallel with all issuers.

Keywords: Anonymous counting tokens; threshold issuance; self-sovereign identity; Sybil-resistance; unlinkability

1 Introduction

Digital identity encompasses online information associated with individuals, businesses, or entities, enabling them to establish and verify their identity in both digital and real-world resources. Currently, digital identities are primarily managed through centralized and federated identity management systems. In centralized identity management, organizations store and control all digital identities within a central server. This typically involves using identifiers like email or username. Federated identity systems rely on single sign-on (SSO) and allow users to utilize a single set of login credentials (from some centralized provider), such as those provided by Facebook or Google, to access multiple applications. This server-controlled identity approach has led to a plethora of catastrophic breaches such as Yahoo’s enormous user account leakage in 2013 [yah17], Facebook’s data breach in 2018 [fac18] and the Microsoft breach in 2023 [ms23] to just name a few.

As a result, there has been an increasing push towards self-sovereign identity (SSI) or decentralized identity systems, leading to a paradigm shift in digital identity management. In this approach, users are liberated from the constraints of a single centralized provider and maintain complete sovereignty over how their identity information is utilized when interacting with online applications and services, thus ensuring they have the ultimate say on how their personal data is shared and used.

Self-Sovereign Identity. In this setting which is also referred to as decentralized identity (DID), users maintain a collection of credentials or decentralized identifiers and decide on their own which information should be revealed to each party. This paradigm shift has sparked numerous initiatives in recent years, with the aim of developing standards that can materialize the vision of self-sovereign identity. Prominent examples include the Decentralized Identity Foundation [dif20], the Decentralized Identifiers working group of W3C [W3C18], and the Sovrin project [Sov18]. While these initiatives have made significant strides, they have yet to address a critical requirement for many real-world systems: Sybil-resistance, or one-person-one-vote, as a fundamental aspect of fairness. In certain scenarios, such as Decentralized Finance (DeFi), the decision-making process heavily relies on collecting votes from system users or stakeholders who have made deposits within the system. Particularly in the context of Decentralized Autonomous Organizations (DAOs) [SKP⁺23], governance is executed through a series of proposals that members vote on via the blockchain. However, in such systems, individuals with a larger possession of governance tokens often wield more voting influence, resulting in a “one-dollar-one-vote” approach. Yet, a more equitable approach is to adopt a “one-person-one-vote” principle, which ensures equal power in decision-making for all users. To realize this aspiration, the establishment of a Sybil-resistant DID system becomes imperative. Such a system

would guarantee a single account for each individual, thereby mitigating the risks associated with identity fraud and enabling fairer participation in decentralized systems. Proof-of-personhood (PoP) [BKKJ⁺17] achieves Sybil-resistance by introducing pseudonym parties as periodic in-person meetings. Pseudonym parties, organized by various entities such as governments and non-profit organizations, serve as a foundation for thwarting adversaries attempting Sybil attacks by verifying the authenticity of individuals and establishing a connection between physical and virtual identities. Nevertheless, the idea’s practicality is challenged by the requirement of periodic physical presence.

CanDID. Recently, Maram et al. [MMZ⁺21] proposed CanDID, which is a DID system achieving Sybil-resistance in a privacy-preserving way without requiring explicit provider-support. To do so, CanDID leverages oracles [ZMM⁺20] to securely relay the rich attested user data present in existing authoritative web servers. Technically, CanDID uses secure multi-party computation (MPC) among a committee of nodes (the so-called CanDID committee) to compute a unique value on a secret-shared deduplication attribute to achieve Sybil-resistance. This value is stored in a table and deduplicated whenever a user requests a new identity to avoid Sybil attacks. The basic design idea behind CanDID is that users can port data from legacy web accounts to create credentials and in particular a master credential (where deduplication happens). Based on this credential, users can be issued context-based credentials for specific purposes. Latter credentials inherit the Sybil-resistance property of the master credential, i.e. only one credential per context is issued.

Notwithstanding its achievements, CanDID does have certain limitations: *(I)* It achieves Sybil-resistance in an expensive way. More precisely, it utilizes MPC among committee nodes on secret shared values of the deduplication attribute which are shared with all committee nodes via an interactive protocol. *(II)* After the deduplication and master credential issuance phases are completed, CanDID requires users to communicate with committee nodes *each time* to collect context-based credentials. These credentials are intended for use in interactions with applications. This approach also contradicts the goal of self-sovereign identity, whose aim is to give users full control over their identities. *(III)* This type of structure requires frequent (complex) interactions, making CanDID vulnerable to a **linkability attack**. Specifically, the reuse of the same pseudonym across multiple transactions to enforce Sybil resistance allows a single malicious committee node to correlate and link transactions initiated by the same user, effectively reducing the privacy guarantees provided by CanDID. Similar vulnerabilities are observed in related works, such as [WGW⁺23]. *(IV)* Since the credentials are not linked to any private key, if the original master credential is stolen or sold on underground online markets, a malicious party can obtain context-based credentials (called application-specific credentials in our work) by presenting the stolen master credential. Consequently, the non-transferability of credentials is at risk.

Before presenting our contributions it is important to note that CanDID considers an identity system as well as a key recovery system. Our focus in this

paper is solely on the former as the latter approach could be directly taken from CanDID or other recovery mechanisms adopted in Web3 [CCK⁺23]. Moreover, in the identity system, CanDID considers the property of accountability, i.e. the CanDID committee can screen users to identify the credentials of suspect users (e.g. those presented on sanction lists). This is realized as a privacy-preserving fuzzy matching via MPC, can also be adopted from CanDID, and is out of the scope of this paper.

Concurrent and independent work. In a recent concurrent and independent work, Crites et al. [CKS24] present an approach that avoids interaction to introduce users to new contexts and requires no state for the issuers. The core idea is achieved via the composition of two verifiable random functions (VRF), cleverly leveraging them to ensure unlinkability while simultaneously linking by pseudonym for all signatures issued within the same context. However, Crites et al. [CKS24] only discuss the proposed Sybil-resistant “SyRA” signature for a single unique attribute representing user’s real-world identity. For attribute-based identification, they discuss in high-level that the context-based statement requested by the verifier can be combined with anonymous attribute-based credentials to demonstrate user attributes in a privacy-preserving way while maintaining resilience against Sybil attacks. Users will obtain keys for both SyRA signature and anonymous attribute-based credentials. This allows subsequently to engage in attribute-based identification with each transcript signed for the context requested by the verifier. Notably, the lower bound for each SyRA signature generation is 12 pairing operations *per attribute*, necessitating repetition for each context or, alternatively, interaction with issuers for each context, akin to CanDID’s methodology, where users submit a set of claims required by the context to receive the per-user per-context key. They leave investigating a concrete efficient design for attribute-based signatures for future work. Our approach in contrast only needs a single per-user call to the tACT algorithm, the most resource-intensive operation, for all attributes required across different contexts in our DID framework (See Section 4 for the implementation results). We thus emphasize a more practice-oriented approach for *multiple* private attribute credentials, built on our new tACT component, which serves as a threshold token issuance mechanism.

1.1 Contributions

Our contribution in this paper is twofold:

Threshold Anonymous Counting Tokens. We introduce the notion of publicly verifiable attribute-based threshold issuance Anonymous Counting Token (tACT). Anonymous Counting Tokens (ACTs), as introduced by Benhamouda et al. in [BRS23], serve as a primitive enabling users to acquire tokens for their chosen private messages, while allowing token issuers or verifiers to impose rate limits on the number of tokens users can obtain or redeem per message. Notably, the approaches proposed in [BRS23] operate within a centralized setting, where a single entity, the issuer, handles token issuance. In contrast, we facilitate the

issuance of ACT tokens in a distributed-trust environment by introducing tACT in a threshold setting. The proposed concept is accompanied by a rigorous formal security model as well as an instantiation that is provably secure in this model. We believe that the concept of tACT might be of independent interest. For our concrete construction we introduce a threshold version of a novel variant of the Shacham-Waters signature scheme [SW13].

DID System Design. We showcase the application of the tACT primitive, by using it as a fundamental building block within our Sybil-resistant Self-Sovereign Identity (S3ID) system, addressing important shortcomings identified in CanDID. These enhancements include:

- *Efficient Deduplication:* Our deduplication process avoids the need for resource-intensive MPC utilized by CanDID [MMZ⁺21]. Instead, it leverages the proposed efficient tACT executed between a user and a group of authorized issuers, each potentially representing distinct entities.
- *Non-interactive application-specific credential generation:* Our solution eliminates the necessity to communicate with the issuers to collect application-specific credentials. Here, the user locally builds application-specific credentials and directly presents them to the verifier. The proposed tACT acts as an anchor empowering users to create *user-issued* unique application-specific anonymous tags to ensure the Sybil-resistance property in direct user-application interactions.
- *Unlinkability:* In contrast to CanDID, where unlinkability breaks in the presence of a single malicious issuer, our architecture maintains unlinkability regardless of corruption of the system entities. We also present a formal definition of strong unlinkability and prove our system security under this stronger definition.
- *Non-Transferability:* The credentials issued within S3ID can only be released by the true owner.

Figure 1 outlines a system flow of our S3ID system and lists important features it achieves. S3ID can operate independently as a standalone service or seamlessly integrate with other self-sovereign identity solutions.

Evaluation. Both constructions are accompanied with implementations in Rust which is the basis for the efficiency evaluation. In comparison to CanDID, we can show that our system is more efficient while at the same time supporting more issuers and stronger security guarantees.

1.2 Technical Overview

Similar to CanDID, we consider a unique user attribute, like the Social Security Number (SSN). This is used as input to a deterministic high-entropy function and is denoted as “a”.

Master credential issuance (tACT token). The user initiates the process by generating a Pedersen commitment $cm = g^r \cdot u^a$ on attribute a which serves as

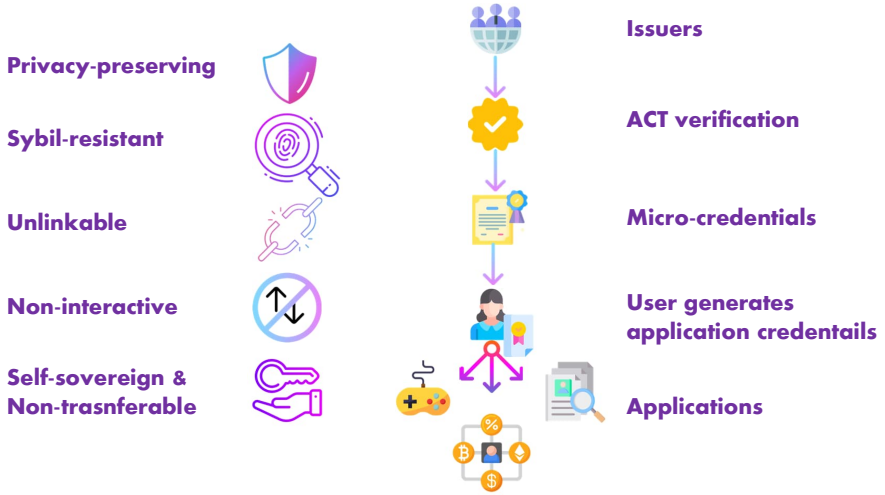


Fig. 1: S3ID system flow (right) and feature list (left).

a blinded request to the set of N issuers, who upon reception generate partial signatures. The user then gathers partial signatures generated by authorized issuers until a threshold t out of N is reached. Then, the user generates a full unblinded signature by combining the collected partial signatures. This unblinded full signature effectively functions as the deterministic high-entropy representation of $\mathbf{a}$ (referred to as tACT), enabling issuers to perform user deduplication. However, a significant challenge arises when attempting to achieve public verifiability of tACT while maintaining the desired properties of unlinkability and anonymity of the attribute. One unsuccessful approach in achieving public verifiability involves revealing a function of randomness r used in the commitment, allowing the verifier to authenticate tACT based on the provided commitment. However, this method introduces a vulnerability where the verifier can conduct guessing attacks to identify the encapsulated attribute $\mathbf{a}$, thus compromising anonymity.

Therefore, we adopt a different verification mechanism inspired by Thyagarajan et al. [TBM⁺20], by combining threshold secret-sharing with a *cut-and-choose* type of argument and utilizing the homomorphic property of the underlying signature. This technique allows us to prove the validity of tACT without disclosing any information regarding the enclosed attribute $\mathbf{a}$. Concretely, the user secret shares the commitment cm using a threshold secret sharing scheme, where a total of n shares are generated, and at least t' shares are required to reconstruct the secret. The first $t' - 1$ shares are determined as $\text{cm}_j := g^{r_j} \cdot u^{a_j}$ for randomly sampled values $(a_j, r_j) \in \mathbb{F}_p$ and $j \in [t' - 1]$. We also set $R_j = \text{pk}^{r_j}$, where pk is the public key of the underlying SW signature scheme [SW13] described in Section 2.2, which is the basis of the tACT construction. The remaining shares are then generated consistently using Lagrange interpolation in the exponent, where each share with index $k \in [t', n]$ is computed based on the t' -th

Lagrange polynomial basis $\ell_{t'}^{[t'-1],k}(\cdot)$ for the points $\{1, \dots, t' - 1, k\}$ and the available $t' - 1$ set of shares:

$$\text{cm}_k = \left(\frac{\text{cm}}{\prod_{j \in [t'-1]} \text{cm}_j^{\ell_j^{[t'-1],k}(0)}} \right)^{\ell_{t'}^{[t'-1],k}(0)^{-1}}.$$

We use $\ell_j^{[t'-1],k}(\cdot)$ as abbreviation for the j -th Lagrange basis polynomial for the points $j \in \{1, \dots, t' - 1\}$. Thereby cm_k is a valid commitment under the corresponding randomness specified below:

$$R_k = \left(\frac{\text{pk}^r}{\prod_{j \in [t'-1]} R_j^{\ell_j^{[t'-1],k}(0)}} \right)^{\ell_{t'}^{[t'-1],k}(0)^{-1}}.$$

The user then sends the commitment shares $\{\text{cm}_k\}_{k \in [1,n]}$ to the issuers who issue partial signatures on the blinded shares of the attribute. The user combines partial signatures to get a full signature σ_k on each cm_k and unblinds σ_k using R_k to get deterministic signature s_k on $\mathbf{a}_k$.

Next, the user employs Lagrange interpolation to reconstruct valid signature s on $\mathbf{a}$ by selecting any set of signature shares s_k of size t' . To verify the legitimacy of s , the verifier randomly chooses a set C consisting of $t' - 1$ elements. In response, the user provides the signature shares $\{s_k\}_{k \in C}$ along with the corresponding random coins $\{R_k\}_{k \in C}$ contained in the commitment shares $\{\text{cm}_k\}_{k \in C}$. The verifier confirms the authenticity of s provided that both the following conditions are met:

1. All $\{s_k\}_{k \in C}$ are valid signatures on the corresponding commitment $\{\text{cm}_k\}_{k \in C}$, i.e. $\forall k \in C : \text{Sig.Verify}(\text{pk}, \text{cm}_k, R_k, s_k) = 1$
2. All commitment shares $\{\text{cm}_{k'}\}_{k' \notin C}$ reconstruct to the commitment cm on attribute $\mathbf{a}$, i.e. $\prod_{k_i \in C} \text{cm}_{k_i}^{\ell_i(0)} \cdot \text{cm}_{k'}^{\ell_{t'}(0)} = \text{cm}$, with $\ell_j = \ell_j^{(k_1, \dots, k_{t'-1}, k')}$ whereas $C = \{k_1, \dots, k_{t'-1}\}$ considered as ordered set.

By collectively satisfying these conditions, we establish that if at least one of the partial signatures outside challenge set C aligns with its respective partial commitment, we can utilize it to reconstruct s . Consequently, a dishonest prover would be required to accurately guess the set C in advance in order to successfully pass the aforementioned verifications without s actually being a valid signature. By selecting suitable values for t' and n , one can ensure that this can only happen with negligible probability. We note that the challenge set C , can be generated non-interactively by the user implementing the Fiat-Shamir transformation [FS86]. A high-level overview of tACT construction is depicted in Figure 2.

Deduplication. Once the tACT token is built through the techniques described above, it will be used as a key ingredient in the S3ID construction.

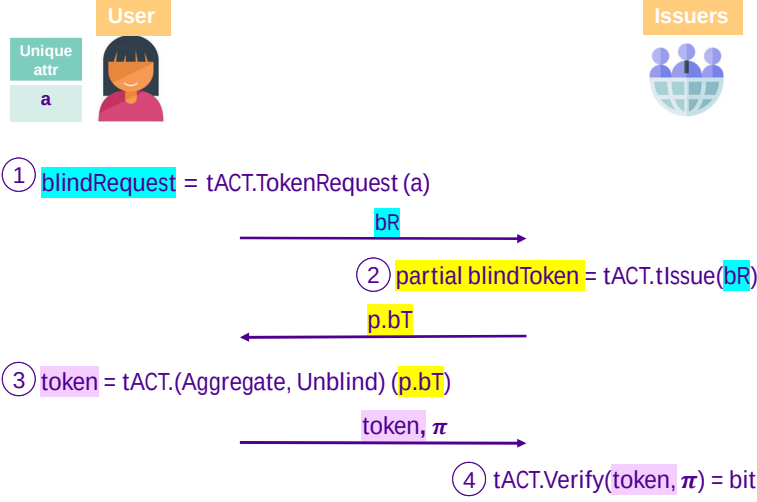


Fig. 2: tACT overview. ① User generates $\text{blindRequest} = \{\text{cm}, \{\text{cm}_k\}_{k \in [1, n]}\}$ and sends this request to the issuers. ② Issuers generate blindToken on the commitment shares. ③ The user collects a threshold number of $\text{partial blindToken}$, combines them to get a full signature, and unblinds the result to acquire the deterministic token . It then generates cut-and-choose proof π and sends (π, token) to the verifier. ④ The verifier checks π to ensure the validity of received token .

Intuitively, in the *deduplication phase* of the S3ID protocol, upon successful verification of the tACT token, the issuers proceed by comparing the token against their redeemed-token database and reject the token if this value already occurs there. Otherwise, update the database by adding redeemed token to it. Besides, as part of the registration process, the user also provides the issuer with a commitment to a uniformly chosen unique key k (represented as cm_k). Commitment cm_k is stored alongside the user's tACT. The key k , known only to the user, facilitates meeting two essential objectives: (I) Non-Transferability: Micro-credentials can be released only by the true owner, (II) Sybil-resistance: The system achieves *one-person-one-ID* while preserving non-interactiveness and strong unlinkability – a property that will be formally defined in Section 3. In the following, we provide a detailed explanation on how these objectives are achieved.

Micro-credential issuance. After completing the deduplication phase, S3ID system enables users to acquire *micro-credentials* for their desired attributes such as age, proof of address, etc., in an anonymous manner. We define a global set of admissible attributes, where each index in the set corresponds only to a specific attribute. To obtain a micro-credential, a user submits a commitment to a specific attribute. Then they proceed to gather sufficient partial signatures from issuers and combine them to form a complete signature which is subsequently unblinded. The issued micro-credential is bound to the unique attribute index

and inherits the Sybil-resistance property of the tACT tokens, i.e. only one micro-credential per user attribute is issued.

Application-specific credentials⁶. With obtained micro-credentials, the user is empowered with a fine-grained disclosure of its attributes in the *application credentials phase*. In particular, when using an application that requires specific attributes, the user locally aggregates the relevant micro-credentials into a compact application credential and shows the aggregated result to the verifying party. The verification of application credentials simultaneously verifies all aggregated micro-credentials in a batch and thus is efficient.

The design of micro-credentials prevents malicious users from obtaining the application credential by replacing a different combination of micro-credentials instead of the designated ones required by the application (*replace attack*). This threat is mitigated by binding each micro-credential to the unique index of its corresponding attribute. During verification, the verifier ensures that the indices in the disclosed micro-credentials align with the required attributes for the application. Moreover, our construction guarantees that only the rightful owner can release the issued micro-credentials (*non-transferability*). This is achieved by using the following technique: issued micro-credentials are *tethered* to the committed unique user key k which prevents malicious users from obtaining the application credential by employing a combination of micro-credentials that do not belong to them. The presence of k in issued micro-credentials ensures that the user must possess k to construct tags during the application credential phase. Thus, users are compelled to build application credentials based on their genuinely owned micro-credentials. This reinforces non-transferability.

The construction so far provides the following desirable features: **(1) Threshold ACT**. The proposed tACT framework facilitates the issuance of ACT tokens in a distributed-trust environment, in contrast to the approach in [BRS23]. **(2) Uniqueness and Fast Deduplication**. User identities are deduplicated, which is highly beneficial for various applications, such as e-voting. Moreover, the deduplication process does not involve costly multi-party computation methods as offered by CanDID [MMZ⁺21]. **(3) Blindness and Attribute-Membership Privacy**. Throughout the entire S3ID processes, issuers are unable to obtain any information regarding user attributes. Besides, issuers are unable to learn whether there is a credential for a specific attribute value within the system. **(4) Non-Transferability**. The credentials can be released only by the true owner. **(5) Batch Verification**. The verification of application credentials simultaneously verifies all aggregated micro-credentials in a batch manner and thus is efficient.

The challenge that we still need to address is limiting the number of submissions per user per application (*Sybil-resistance*), while preserving strong unlinkability and non-interactiveness. CanDID employs an approach mitigating the Sybil attack, where a credential for an application with a unique application name is issued corresponding to the unique pseudonym of the user initiating

⁶ We recall that CanDID calls them context-based credentials.

each query. However, this method presents two significant problems: Firstly, it exhibits interactivity as it requires user engagement with a threshold number of issuers to obtain the necessary application credentials before connecting with the verifier. The interactive nature of this process can lead to system inefficiencies. Secondly, and more importantly, this method compromises the principle of unlinkability. By learning the user’s unique pseudonym, issuers can establish links between different user transactions. This violation of unlinkability undermines the intended level of privacy and anonymity in the system. Therefore, despite its initial attempt to address the Sybil attack, the CanDID approach encounters two significant problems: interactivity and the compromise of unlinkability.

Adding Sybil-resistance. Our proposed system introduces a user-empowered solution to ensure Sybil-resistance by enabling users to generate unique, application-specific anonymous tags based on the *unique* application message l for use in direct user-application interactions. During the redemption process, the user presents the unique tag along with the corresponding application credential to the relying party. The verifier then checks if the tag for the same application message l and the same user has already been redeemed by checking its database, preventing double-submission, and subsequently examines the application credential. However, a challenge arises during this redemption check, as it is crucial to remove any link to the user’s identity while retaining the ability to verify that the tag was generated by a user-registered key k . To overcome this challenge, we adopt an approach close to the work presented by Benhamouda et al. [BRS23] by employing a pseudorandom function (PRF) as the token issuance mechanism. However, in our case, the main difference is that users may compute this PRF by themselves rather than relying on an issuer. The PRF is evaluated under the user-registered key k on the unique application message l . The authors note that this is related to *scope-exclusive pseudonyms*, a concept employed in anonymous credential systems [CDL⁺13]. The adopted PRF has algebraic properties that enable it to seamlessly integrate with efficient zero-knowledge proofs based on Sigma protocols and can be made non-interactive using the Fiat-Shamir heuristic for the proof of correct evaluation. Therefore, the user efficiently demonstrates the correct evaluation of the tag on the message with respect to the key k encapsulated in all user micro-credentials, in a zero-knowledge manner. This can be achieved through two different PRF constructions: $\text{PRF}(k, l) = H(l)^k$, or $\text{PRF}(k, l) = g^{(k+l)^{-1}}$, the former being the Naor-Pinkas-Reingold PRF [NPR99] and the latter being the Dodis-Yampolskiy PRF [DY05]. Since l is known by the verifier, the former PRF can also be efficiently proven in zero-knowledge. By employing this mechanism, users gain the ability to generate unique application-specific tags that can be verified with an efficient proof of correct evaluation. As a result, in addition to previously mentioned features, the system achieves the (6) **Sybil-resistance** property in a (7) **non-interactive** and (8) **unlinkable** manner. An overview of our S3ID identity system is depicted in Figure 3. We also provide a high-level comparison of our construction over CanDID in Table 1.

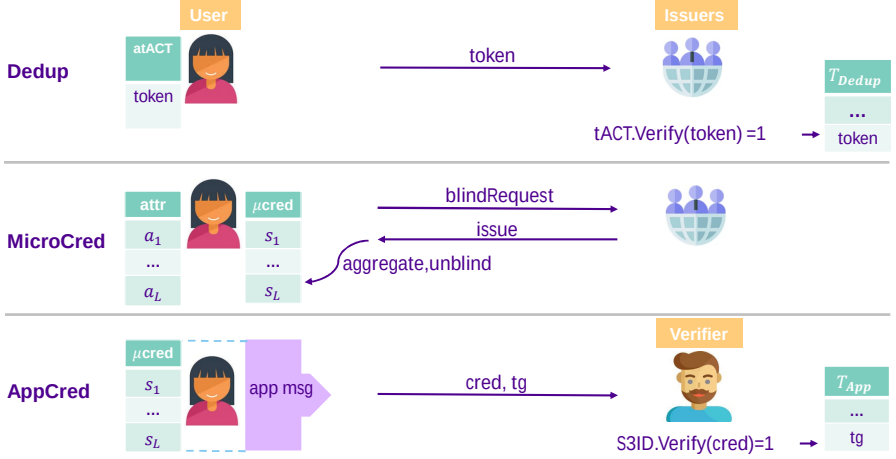


Fig. 3: S3ID overview. ① **Dedup**: User submits tACT token to the issuers, who verify the token and compare it against their database T_{Dedup} and reject if $token \in T_{Dedup}$. Otherwise, they update T_{Dedup} by adding token, cm_k to it. ② **MicroCred**: User interacts with the issuers to obtain micro-credentials $\{s_i\}_{i=1}^L$ on her attributes $\{a_i\}_{i=1}^L$. ③ **AppCred**: For an application with a unique message, the user locally aggregates micro-credentials encapsulating a subset of attributes required by the application and, as a result, shows the succinct application credential value to the verifying entity. In addition, the user generates a tag under the registered key k on the unique application message to enable limiting double submissions by the verifier. The whole AppCred phase is executed locally on the user’s device and is devoid of any interactions involving the issuers.

Table 1: Comparison of our construction over CanDID [MMZ+21] identity system.

Features	unf.	blind.	Sybil-resistance	unlink.	non-transfer.	non-interact. [‡]
CanDID [MMZ+21]	✓	✓ [†]	✓	✗	✗	✗
S3ID	✓	✓	✓	✓	✓	✓

[†] CanDID is pseudonymous rather than being fully anonymous; i.e., the issuer learns a unique pseudonym as the identifier of a user who initiates each query and is associated with that user’s CanDID credentials.

[‡] This property is with respect to the AppCred phase. Dedup and MicroCred phases are intrinsically interactive, akin to the CanDID system.

1.3 Application Domains

In general, DID systems are a key-enabling technology for today’s and tomorrow’s distributed world. Our lives increasingly depend on digital services, where centralized identity mechanisms are becoming an increasingly serious threat to our free society. A cryptographically secure and efficient DID mechanism that

is Sybil-resistant and unlinkable and can cope with actual threats of identity theft and privacy breach, as developed within S3ID, helps to protect individual privacy and secure personal data. Here, we exemplify applications of interest that our S3ID construction could be beneficial to.

Proof of Unique Personhood. S3ID can serve as evidence of a person’s distinct human identity and distinguish them from bots or other automated entities.

Passwordless Authentication. A blockchain wallet integrated with S3ID enables users to effortlessly access any portal using a single set of login credentials, representing an evolved form of single-sign-on (SSO).

Simple Know Your Customer (KYC). The DID for each user can be reused on different platforms for user onboarding purposes on various DeFi Decentralized Applications (DApps) without incurring the expenses associated with performing KYC for each individual platform.

Reputation and Credit Scoring Systems. In contrast to Web 2.0, using DID, Web 3.0 users have the ability to transfer and aggregate their reputation data across multiple DApps, without the need to make a fresh start each time.

Real-world Asset Tokenization: S3ID can be instrumental in the process of tokenizing real-world assets, ensuring a one-to-one correspondence between the physical asset itself and the token issued on-chain.

1.4 Preliminaries

We recall some notions that are required for our constructions. Our notation, Pedersen commitments, zero-knowledge proofs of knowledge (ZKPoK), and oracle systems are discussed in Appendix B.

2 Attribute-Based Threshold Issuance Anonymous Counting Tokens

2.1 Definitions and Security Properties

Now, we introduce the formal definitions for attribute-based threshold anonymous counting tokens (tACTs) and present the security requirements they must meet.

Definition 1 (tACT). *An attribute-based threshold anonymous counting token (tACT) scheme comprises a set of algorithms that are defined as follows (public parameters $\mathbf{pp}$ are assumed to be an implicit input to all algorithms):*

- $(\mathbf{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda, t, N)$: On input security parameter 1^λ , and parameters t, N representing the threshold and overall number of issuers it outputs system public parameters $\mathbf{pp}$ and private parameters⁷ prv_j for the token issuer $I_j, j \in [1, N]$.

⁷ This is usually run via a distributed key generation protocol.

- $(\text{st}_{\text{Rg}}, \text{cm}) \leftarrow \text{Register}(\mathbf{a})$: On input the public parameters pp for the $t\text{ACT}$ scheme and unique user attribute $\mathbf{a}$, the user outputs a commitment $\text{cm} = \text{Commit}(\mathbf{a}; r)$ and a registration state st_{Rg} including $\mathbf{a}, r$.
- $(\text{blindRequest}, \text{rand}_{\mathbf{a}}) \leftarrow \text{TokenRequest}(\text{st}_{\text{Rg}}, \text{cm})$: This algorithm is run by the user, on input commitment cm and registration state st_{Rg} outputs a blinded token request denoted as blindRequest (including cm) and accompanying state information $\text{rand}_{\mathbf{a}}$ (including st_{Rg}).
- $\text{blindToken}_j \leftarrow \text{tIssue}(\text{blindRequest}, \text{prv}_j)$: Is run by issuer I_j holding private prv_j , by which the user obtains a blinded partial token embedding the private attribute $\mathbf{a}$.
- $\text{blindToken} \leftarrow \text{Aggregate}(\{\text{blindToken}_j\}_{j \in [t]})$: Is run by the user to combine any subset of t partial tokens into a blinded token blindToken .
- $\text{token} \leftarrow \text{Unblind}(\text{blindToken}, \text{rand}_{\mathbf{a}})$: The user, given the blind token blindToken and the corresponding state information $\text{rand}_{\mathbf{a}}$ generated during the TokenRequest phase for attribute $\mathbf{a}$, runs this algorithm to generate the unblinded token token specific to the private attribute $\mathbf{a}$.
- $\pi \leftarrow \text{Prove}(\text{token}, \text{aux}, \text{rand}_{\mathbf{a}})$: Is run by user and inputs the token token embedding the private attribute $\mathbf{a}$, auxiliary information aux , and its corresponding state information $\text{rand}_{\mathbf{a}}$, and outputs a proof π .
- $\{0, 1\} \leftarrow \text{Verify}(\text{token}, \pi, \text{blindRequest})$: On input the token token , proof π , and blinded token request blindRequest , outputs a bit $b = 1$ if the verifier accepts token as a valid token with respect to the blindRequest , and $b = 0$ otherwise. We note that during the verification process, the verifier remains completely unaware of any information regarding attribute $\mathbf{a}$.

Properties. A $t\text{ACT}$ system is called secure if it is correct, unforgeable, and unlinkable as formally defined below:

Definition 2 (Correctness). A $t\text{ACT}$ scheme is called correct if any token that is produced by honestly following the $t\text{ACT}$ algorithms can be successfully verified, i.e. for any sets of system and issuers' parameters $(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda)$, all attributes $\mathbf{a}$ and all state information $\text{st}_{\text{Rg}}, \text{rand}_{\mathbf{a}}, (\text{st}_{\text{Rg}}, \text{cm}) \leftarrow \text{Register}(\mathbf{a})$ $(\text{blindRequest}, \text{rand}_{\mathbf{a}}) \leftarrow \text{TokenRequest}(\text{st}_{\text{Rg}}, \text{cm})$, $\text{blindToken}_j \leftarrow \text{tIssue}(\text{blindRequest}, \text{prv}_j)$, $\text{blindToken} \leftarrow \text{Aggregate}(\{\text{blindToken}_j\}_{j=1}^t)$, $\text{token} \leftarrow \text{Unblind}(\text{blindToken}, \text{rand}_{\mathbf{a}})$, $\pi \leftarrow \text{Prove}(\text{token}, \text{aux}, \text{rand}_{\mathbf{a}})$, the verification algorithm $\text{Verify}(\text{token}, \pi, \text{blindRequest})$ outputs 1 with overwhelming probability.

Adversary classes. The unforgeability definition encompasses a category of adversaries denoted by $\mathcal{A}$ who can corrupt up to $t - 1$ out of N issuers. We note that while for conventional threshold signatures it might be desirable to support adaptive security for very large N [CKM23], in our applications N can be assumed to be reasonably small, i.e., having a few issuers, and thus we can obtain adaptive security for free via a simple guessing argument which incurs a $\binom{N}{t}$ loss. Additionally, $\mathcal{A}$ holds the ability to corrupt any number of users within the system. For unlinkability, $\mathcal{A}$ can corrupt *any* number of issuers and verifiers.

Unforgeability. This property ensures that an adversary controlling issuing entities in corrupted set CS cannot generate tokens for more attributes than the ones for which it has requested tokens. Furthermore, it cannot generate more than one token for a specific attribute per user.

Definition 3 (Unforgeability). A $t\text{ACT}$ scheme is unforgeable if for any PPT adversary $\mathcal{A}$ and any $T \geq 0, R \geq 0$

$$\text{Adv}_{t\text{ACT}, \mathcal{A}}^{UF}(\lambda) := \Pr[UF_{t\text{ACT}, \mathcal{A}}(\lambda) \rightarrow 1] \leq \text{negl}(\lambda),$$

where the experiment $UF_{t\text{ACT}, \mathcal{A}}(\lambda)$ is defined in [Figure 4](#).

Game $UF_{t\text{ACT}, \mathcal{A}}(\lambda)$	$\mathcal{O}_{\text{Register}}(\text{cm})$
$(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda)$ $\text{flag} = 0, T = 0, R = 0, \text{CS} = \emptyset$ $\text{HS} := [1, N] \setminus \text{CS}$ $\text{CS}, \{\mathbf{a}_i, r_i, \text{token}_i, \pi_i, \mathbf{bR}_i\}_{i \in [S+1]}$ $\leftarrow \mathcal{A}^{\mathcal{O}_{\text{Register}}, \mathcal{O}_{\text{tIssue}}, \mathcal{O}_{\text{Corrupt}}}(\text{pp})$ $S := \left\lfloor \frac{T}{(t - \text{CS})} \right\rfloor$ return $ \text{CS} < t \wedge$ $\left(\begin{array}{l} \text{//Type-1 forgery:} \\ \forall i, j \in [S+1], \mathbf{a}_i \neq \mathbf{a}_j \wedge \\ \text{Verify}(\text{token}_i, \pi_i, \mathbf{bR}_i) = 1 \\ \text{or} \\ \text{//Type-2 forgery:} \\ \exists i, j \in [S+1] : S \geq R \wedge \\ \mathbf{a}_i = \mathbf{a}_j \wedge \\ \text{token}_i \neq \text{token}_j \wedge \\ \text{cm}_i = \text{Commit}(\mathbf{a}_i, r_i) \wedge \{\text{cm}_i \in \mathbf{bR}_i\} \\ \text{cm}_j = \text{Commit}(\mathbf{a}_j, r_j) \wedge \{\text{cm}_j \in \mathbf{bR}_j\} \\ \text{Verify}(\text{token}_i, \pi_i, \mathbf{bR}_i) = 1 \wedge \\ \text{Verify}(\text{token}_j, \pi_j, \mathbf{bR}_j) = 1 \end{array} \right)$	$\text{state} = \text{state} \cup \text{cm}$ $R = R + 1$ $\mathcal{O}_{\text{tIssue}}(j, \mathbf{bR})$ Assert ($j \in \text{HS}$) $T = T + 1$ $\text{blindToken}_j \leftarrow$ $\text{tIssue}(\mathbf{bR}, \text{prv}_j)$ $\text{flag} = 1$ return blindToken_j $\mathcal{O}_{\text{Corrupt}}(k)$ if $\text{flag} = 1$ or $k \in \text{CS}$ then abort else $\text{CS} = \text{CS} \cup \{k\} \wedge$ $\text{HS} = \text{HS} \setminus \{k\}$ return prv_k

Fig. 4: Unforgeability experiment for a $t\text{ACT}$ scheme.

Unlinkability. This property ensures that the adversary $\mathcal{A}$ who has the ability to corrupt all issuers cannot link user token requests or unblinded tokens with the private user attribute. [Figure 5](#) formally defines the game $\text{UNLINK}_{t\text{ACT}, \mathcal{A}}(\lambda)$. The intuition behind this definition is as follows. The adversary $\mathcal{A}$ that has corrupted all issuers, can instruct users in the system to generate blind requests $\mathbf{bR}$ for attributes of its choice through the $\mathcal{O}_{\text{TokenRequest}}$ oracle. The adversary should

distinguish blind requests $\mathbf{bR}$ (oracle $\text{ChI}_{\text{Issue}}$) for one of two adversary chosen attributes; or it needs to distinguish unblinded tokens generated by the $\mathcal{O}_{\text{Unblind}}$ oracle. The definition requires that tACT.Unblind and tACT.Verify succeed on both the blind tokens provided by the adversary.

Definition 4 (Unlinkability). A tACT scheme is unlinkable if for PPT adversary $\mathcal{A}$ the advantage defined as

$$\text{Adv}_{\text{tACT}, \mathcal{A}}^{\text{UNLINK}}(\lambda) := |2 \Pr[\text{UNLINK}_{\text{tACT}, \mathcal{A}}(\lambda) \rightarrow 1] - 1|,$$

where the experiment $\text{UNLINK}_{\text{tACT}, \mathcal{A}}(\lambda)$ is defined in [Figure 5](#), is negligible, i.e.,

$$\text{Adv}_{\text{tACT}, \mathcal{A}}^{\text{UNLINK}}(\lambda) \leq \text{negl}(\lambda).$$

Game $\text{UNLINK}_{\text{tACT}, \mathcal{A}}(\lambda)$	$\text{ChI}_{\text{tIssue}}(\mathbf{a}_0, \mathbf{a}_1)$
$(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda, t, N)$ $\mathcal{Q} = \emptyset$ $\mathbf{b} \xleftarrow{\$} \{0, 1\}$ $\mathbf{b}' \leftarrow \mathcal{A}^{\text{GetPrm}, \mathcal{O}_{\text{TokenRequest}}, \text{ChI}_{\text{tIssue}}, \mathcal{O}_{\text{Unblind}}}(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]})$ return $(\mathbf{b}' == \mathbf{b})$	if $\mathbf{a}_0$ is from GetPrm or if $\mathbf{a}_1$ is from GetPrm or $\mathbf{a}_0 = \mathbf{a}_1$ then abort $(\text{st}_{\text{Rg}}, \text{cm}) \leftarrow \text{Register}(\mathbf{a}_{\mathbf{b}})$ $(\mathbf{bR}, r) \leftarrow \text{TokenRequest}(\text{cm}, \text{st}_{\text{Rg}})$ return $\mathbf{bR}$
$\text{GetPrm}(\mathbf{a})$ $(\text{st}_{\text{Rg}}, \text{cm}) \leftarrow \text{Register}(\mathbf{a})$ return $\text{cm}, \text{st}_{\text{Rg}}$ $\mathcal{O}_{\text{TokenRequest}}(\text{cm}, \text{st}_{\text{Rg}})$ if cm not from GetPrm or $(\text{cm}, *) \in \mathcal{Q}$ then abort $(\mathbf{bR}, r) \leftarrow \text{TokenRequest}(\text{cm}, \text{st}_{\text{Rg}})$ $\mathcal{Q} = \mathcal{Q} \cup (\mathbf{bR}, r)$ return $\mathbf{bR}$	$\mathcal{O}_{\text{Unblind}}(\mathbf{bR}_0, \mathbf{bR}_1, \mathbf{bT}_0, \mathbf{bT}_1)$ if $(\mathbf{bR}_0, *) \notin \mathcal{Q}$ or $(\mathbf{bR}_1, *) \notin \mathcal{Q}$ abort $\text{Find}(\mathbf{bR}_{\mathbf{b}}, r_{\mathbf{b}}) \in \mathcal{Q}$ $\text{tkn} \leftarrow \text{Unblind}(\mathbf{bT}_{\mathbf{b}}, r_{\mathbf{b}})$ return tkn

Fig. 5: Unlinkability experiment for a tACT scheme.

2.2 A (Threshold) SW Variant

For our construction we propose a novel variant of the Shacham-Waters (SW) signatures [[SW13](#)], which are a variant of BLS signatures [[BLS01](#)], but in contrast

to BLS allows for the homomorphic aggregation of signed messages. Specifically, the scheme in [SW13], referred to as SW in this paper, signs messages (i, m) instead of just m , such that the scheme is homomorphic with respect to the m but not the i part. We extend SW to construct a threshold version of the signature scheme, denoted as tSW. For consistency with the original scheme [SW13] and the TS-UF-1 security result for threshold BLS [BL22], which we rely on in our security analysis, we describe our scheme in the symmetric pairing setting and note that any such protocol can be (automatically) translated to the asymmetric setting [AGOT14, AGH15, AHO16]. In the following, we describe a threshold version tSW of our variant.

- $\text{pp} \leftarrow \text{tSW.Setup}(1^\lambda)$: Let $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ be a bilinear group of prime order p , where p is a λ bit prime, and g and u are generators of $\mathbb{G}_1$. Let $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ be the BLS hash. The algorithm outputs system public parameters $\text{pp} = (e, p, \mathbb{G}_1, \mathbb{G}_2, g, u, H)$.
- $(\text{pk}, \{\text{sk}_j\}_{j \in [1, N]}) \leftarrow \text{tSW.KeyGen}(1^\lambda, t, N)$: The parties execute a distributed key generation protocol [KHG12] to generate BLS secret signing key sk where party $j \in [1, N]$ receives the share sk_j of sk such that t shares of the secret reconstruct sk . The public key is set to $\text{pk} = g^{\text{sk}}$.
- $\sigma_j \leftarrow \text{tSW.Sign}(\text{sk}_j, (i, m))$: On input secret key sk_j and message (i, m) , outputs a partial signature $\sigma_j = (H(i) \cdot u^m)^{\text{sk}_j}$.
- $\sigma \leftarrow \text{tSW.Comb}(\{\sigma_c\}_{c \in C})$: On input t partial signatures $\{\sigma_c\}_{c \in C}$ where $C = \{c_1, \dots, c_t\} \subset [N]$ considered as ordered set, outputs a full signature $\sigma = \prod_{c_j \in C} \sigma_{c_j}^{\ell_j^{(0)}}$ with ℓ_j being the j^{th} Lagrange polynomial for points $c_1, \dots, c_t$.
- $\{0, 1\} \leftarrow \text{tSW.Verify}(\text{pk}, (i, m), \sigma)$: On input a public key pk , message (i, m) , and signature σ checks whether $e(\sigma, g) = e(H(i) \cdot u^m, \text{pk})$ and outputs a bit $b \in \{0, 1\}$.

Remark. We note that the tSW.Sign algorithm instead of signing commitment u^m can equivalently take a Pedersen commitment $\text{cm} = \text{Ped.Commit}(m; r) = g^r u^m$ as input⁸, and output $\sigma_j = (H(i) \cdot \text{cm})^{\text{sk}_j}$ as partial signature. In such case, we call it $\text{tSW}_{\text{Ped}}.\text{Sign}$. Then, given a signature σ under $\text{tSW}_{\text{Ped}}.\text{Sign}$ for commitment $\text{cm} = g^r u^m$ on message (i, m) , $\text{tSW}_{\text{Ped}}.\text{Verify}(\text{pk}, \text{cm}, i, \sigma)$ checks whether $e(\sigma, g) = e(H(i) \cdot \text{cm}, \text{pk})$. Furthermore, note that one can publicly convert between signatures of the two schemes. In particular, given a signature σ under tSW_{Ped} for $g^r u^m$ one can obtain signature σ' under tSW by setting $\sigma' = \sigma \cdot \text{pk}^{-r}$.

As demonstrated in [SW13], such signatures can be efficiently aggregated across different messages, simplifying the verifier's task through batch verification, where all aggregated signatures can be simultaneously verified in a single round.

For the security of tSW, we refer to Appendix C.1.

⁸ It can also take a generalized Pedersen commitment [Ped91].

2.3 tACT Construction

Now we present our tACT construction. Consider a unique user attribute denoted as $\mathbf{a}$, e.g., the Social Security Number (SSN). In order to obtain the token, we will need to obviously evaluate a deterministic high-entropy function F of $\mathbf{a}$ in a distributed manner, where the issuers share the secret key $\mathbf{sk}$, while the user has the message $(\mathbf{a}, r_{\mathbf{a}})$ as input. Upon completion of the protocol, the user is provided with the output $F(\mathbf{a}, r_{\mathbf{a}})$, while the issuers gain no information. We further require $F(\mathbf{a}, r_{\mathbf{a}})$ to be *anonymously* verifiable; i.e., a public verifier can verify the token without learning anything about $\mathbf{a}$. The key ingredients for constructing tACT are Pedersen commitments $\text{Ped} = (\text{Setup}, \text{Commit})$, and threshold SW signature $\text{tSW}_{\text{Ped}} = (\text{KeyGen}, \text{Sign}, \text{Comb}, \text{Verify})$. We also let n be a statistical security parameter and set $t' = \frac{n}{2} + 1$, with (t', n) used as parameters for the cut-and-choose. Our tACT scheme is presented in [Algorithm 1](#). In the present scenario, tACT construction integrates the throttling property during the redemption process, which prevents a user from redeeming a token multiple times.

It begins with the issuers executing a distributed key generation protocol [[KHG12](#)] to generate shares of SW secret signing key for each issuer $j \in [1, N]$ and publish public key $\mathbf{pk}$. Next, in `tACT.Register`, a user generates a commitment $\mathbf{cm}$ to a unique attribute $\mathbf{a}$. The user then secret shares the commitment using a (t', n) threshold secret sharing scheme in the `tACT.TokenRequest` algorithm. These partial commitments serve as `blindRequest`, upon which the issuers generate partial `blindTokens` in the `tACT.tIssue` algorithm. The user then gathers threshold many partial signatures generated by authorized issuers. These partial signatures are then combined to form a complete signature, which is then unblinded (`tACT.(Aggregate, Unblind)`). The resulting unblinded full signature is deterministic high-entropy function of $\mathbf{a}$, referred to as `token`.

To prove the validity of `token` in the `Prove` algorithm while preserving attribute anonymity, the user non-interactively determines a set C containing $t' - 1$ elements. The user then provides the unblinded shares $\{s_k\}_{k \in C}$ along with the corresponding random coins contained in the commitment shares $\{\mathbf{cm}_k\}_{k \in C}$. The verifier confirms the authenticity of `token` provided that the conditions in the `Verify` algorithm are collectively satisfied.

For the security analysis of tACT, we refer to [Appendix C.2](#).

Algorithm 1 tACT Construction

- 1: **tACT.Setup**($1^\lambda, t, N$)
 - 2: Run $\text{pp}' \leftarrow \text{tSW.Setup}(1^\lambda)$ and let $\text{H}' : \{0, 1\}^* \rightarrow \{C \subset [1, n] \mid |C| = t' - 1\}$, where (t', n) are parameters of cut-and-choose. All hashes are treated as random oracle. Let $\mathbb{I} = \{I_1, \dots, I_N\}$ denote the set of authorized issuers.
 - 3: The issuers run $(\text{pk}, \{\text{sk}_j\}_{j \in [1, N]}) \leftarrow \text{tSW.KeyGen}(1^\lambda, t, N)$, where $I_j \in \mathbb{I}$ receives the share sk_j of tSW secret signing key sk and public key is set to $\text{pk} = g^{\text{sk}}$.
 - 4: **Output:** $\text{pp} \leftarrow (\text{pp}', \text{H}', \text{pk})$, $\text{prv}_j \leftarrow \text{sk}_j$
 - 5: **tACT.Register**(a)
 - 6: Generate a commitment on a unique attribute a defined by the system at index i^* of all admissible attributes. Set $\text{cm} = \text{Ped.Commit}(\text{a}; r)$, where $r \leftarrow \$_{\mathbb{F}_p}$.
 - 7: **Output:** $\text{st}_{\text{Rg}} = \{\text{a}, r\}$, cm
 - 8: **tACT.TokenRequest**($\text{st}_{\text{Rg}}, \text{cm}$)
 - 9: For all $k \in [t' - 1]$ uniformly sample $(\text{a}_k, r_k) \in \mathbb{F}_p$ then set $\text{cm}_k = \text{Ped.Commit}(\text{a}_k; r_k)$ and $R_k = \text{pk}^{r_k}$.
 - 10: For all $k \in [t', n]$, compute $\text{cm}_k = \left(\text{cm} \prod_{j \in [t' - 1]} \text{cm}_j^{-\ell_j^{[t' - 1], k}(0)} \right)^{\ell_j^{[t' - 1], k}(0)^{-1}}$,

$$R_k = \left(\text{pk}^r \prod_{j \in [t' - 1]} R_j^{-\ell_j^{[t' - 1], k}(0)} \right)^{\ell_j^{[t' - 1], k}(0)^{-1}} \quad \text{where } \ell_j^{[t' - 1], k} \text{ is the } j\text{-th Lagrange}$$
basis polynomial for the points $1, \dots, t' - 1, k$.
 - 11: User samples a key $k \leftarrow \$_{\mathbb{F}_p}$ and outputs $\text{cm}_k = \text{Ped.Commit}(k; r_k)$, where $r_k \leftarrow \$_{\mathbb{F}_p}$
 - 12: **Output:** $\text{blindRequest} \leftarrow \{\text{cm}, \{\text{cm}_k\}_{k \in [1, n]}, \text{cm}_k\}$, $\text{rand}_a \leftarrow \{\text{st}_{\text{Rg}}, \{R_k\}_{k \in [1, n]}, k, r_k\}$
 - 13: **tACT.tlssue**($\text{blindRequest}, \text{prv}_j$)
 - 14: **Each** $I_j \in \mathbb{I}$:
 - a: Parse $\text{blindRequest} = \{\text{cm}, \{\text{cm}_k\}_{k \in [1, n]}, \text{cm}_k\}$
 - b: Check if $\text{cm} = \prod_{k \in [n]} \text{cm}_k^{\ell_k^{[n]}(0)}$ where $\ell_j^{[n]}$ is the j -th Lagrange basis polynomial for points $[n]$
 - c: Compute σ_{jk} as $\sigma_{jk} = \text{tSW}_{\text{Ped}}.\text{Sign}(\text{sk}_j, \text{cm}_k, i^*)$ and set $\text{blindToken}_{jk} \leftarrow \sigma_{jk}$
 - 15: **Output:** $\{\text{blindToken}_{jk}\}_{k \in [1, n]}$
 - 16: **tACT.(Aggregate, Unblind)**($\{\text{blindToken}_{jk}\}_{j \in [t], k \in [1, n]}, \text{rand}_a$)
 - 17: Collect a threshold t number of partial blindToken_{jk} on each a_k
 - 18: Combine partial signatures: $\sigma_k = \text{tSW.Comb}(\{\sigma_{jk}\}_{j \in [t]})$
 - 19: Unblind σ_k by computing $s_k = \sigma_k \cdot R_k^{-1}$
 - 20: Compute $s = \prod_{k \in [t']} s_k^{\ell_k^{[t']}(0)}$ where $\ell_j^{[t']}$ denotes the j -th Lagrange basis polynomial for the points $[t']$.
 - 21: **Output:** $\text{token} \leftarrow s, \{s_k\}_{k \in [1, n]}$
 - 22: **tACT.Prove**($\text{token}, \{s_k\}_{k \in [1, n]}, \text{rand}_a$)
 - 23: Compute $C \leftarrow \text{H}'(\text{token})$
 - 24: Set $\{s'_k = s_k^k\}_{k \in [1, n]}$, $\{R'_k = R_k^k\}_{k \in C}$, and $\text{pk}' = \text{pk}^k$
 - 25: Generate $\pi = \text{ZKPoK}\{(\text{a}, k, r, r_k) : \text{cm} = \text{Ped.Commit}(\text{a}; r) \wedge \text{cm}_k = \text{Ped.Commit}(k; r_k) \wedge \text{pk}' = \text{pk}^k\}$
 - 26: **Output:** $\text{token}, \pi_{\text{token}} \leftarrow \{\{s'_k\}_{k \in [1, n]}, C, \{R'_k\}_{k \in C}, \text{pk}', \pi\}$
-

27: **tACT.Verify**(token, π_{token} , blindRequest)
28: Parse $\pi_{\text{token}} = \{\{s'_k\}_{k \in [1,n]}, C, \{R'_k\}_{k \in C}, \text{pk}', \pi\}$
29: If any of the following conditions is met, set bit $\leftarrow 0$. Otherwise, set bit $\leftarrow 1$.
a: $e(\text{token}, \text{pk}') \neq e(\prod_{k \in [t']} s'^{\ell_k^{[t']}(0)}, \text{pk})$ where $\ell_j^{[t']}$ is the j -th Lagrange basis polynomial for points $[t']$ {to ensure the token's well-formedness.}
b: $\text{ZKVerify}((\text{cm}, \text{cm}_k, \text{pk}'), \pi) = 0$
c: $C \neq H'(\text{token})$ {cut-and-choose verification.}
d: $\exists k' \notin C : \prod_{c_j \in C} \text{cm}_{c_j}^{\ell_j^{C,k'}(0)} \cdot \text{cm}_{k'}^{\ell_{t'}^{C,k'}(0)} \neq \text{cm}$ where $\ell_j^{C,k'}$ denotes the j -th Lagrange basis polynomial for the points $c_1, \dots, c_{t'-1}, k'$ where $C = \{c_1, \dots, c_{t'-1}\}$.
e: $\exists k \in C : \text{tSW}_{\text{Ped}}.\text{Verify}(\text{pk}', \text{cm}_k, i^*, R'_k \cdot s'_k) = 0$
30: **Output:** bit

3 Building a DID System

3.1 Definitions and Properties

Subsequently, we formalize a Sybil-Resistance Self-Sovereign Identity (S3ID) system and provide a precise definition of its essential security properties.

Definition 5 (S3ID). *A Sybil-resistant self-sovereign identity scheme comprises a set of algorithms that are defined as follows (public parameters pp are assumed to be an implicit input to all algorithms):*

- $(\text{pp}, \{\text{prv}_j\}_{j \in [1,N]}) \leftarrow \text{Setup}(1^\lambda, t, N)$: On input security parameter 1^λ , and t, N representing the threshold and overall number of issuers it outputs system public parameters pp and private parameters⁹ prv_j for issuer $I_j, j \in [1, N]$.
- $(\text{pp}_U, \text{prv}_U, \text{T}_{\text{Dedup}}) \leftarrow \text{Dedup}(\mathbf{a}, \{\text{prv}_j\}_{j \in [1,N]}, \text{T}_{\text{Dedup}})$: Is an interactive protocol between a user and the issuers. User input is private unique attribute $\mathbf{a}$. Issuers inputs are private parameters $\{\text{prv}_j\}_{j \in [1,N]}$. This algorithm generates user public and private parameters respectively denoted as pp_U and prv_U . In addition, the system updates the deduplication table T_{Dedup} , which serves as a database of redeemed tokens. This table helps to ensure that each token can only be redeemed once.
- $(\{\mathbf{s}_i\}_{i=1}^L, \text{T}_{\text{Dedup}}) \leftarrow \text{MicroCred}(\{\mathbf{a}_i\}_{i=1}^L, \text{pp}_U, \text{prv}_U, \{\text{prv}_j\}_{j \in [1,N]}, \text{T}_{\text{Dedup}})$: Is an interactive protocol between a user and the issuers. User inputs are private attributes $\{\mathbf{a}_i\}_{i=1}^L$, and private parameters prv_U . Issuers inputs are private parameters $\{\text{prv}_j\}_{j \in [1,N]}$ and user public parameters pp_U . The user obtains a set of micro-credentials $\{\mathbf{s}_i\}_{i=1}^L$ each embedding one of the private attributes $\{\mathbf{a}_i\}_{i=1}^L$.
- $(\text{cred}, \text{tg}) \leftarrow \text{AppCred}(\{\mathbf{a}_i\}_{i=1}^L, \{\mathbf{s}_i\}_{i=1}^L, \text{prv}_U, l, \phi)$: This algorithm is run by the user who inputs private attributes $\{\mathbf{a}_i\}_{i=1}^L$, micro-credentials $\{\mathbf{s}_i\}_{i=1}^L$, private

⁹ This is usually run via a distributed key generation protocol.

parameters prv_U , unique application message l , and application-specific statement ϕ . It outputs a concise credential cred certifying the satisfaction of ϕ by a subset of private attributes $\{\mathbf{a}_i\}_{i=1}^L$. Additionally, the user creates a tag tg which is unique per user per application and serves the purpose of enforcing the Sybil-resistance property.

- $(\{0, 1\}, \mathsf{T}_{\text{App}}) \leftarrow \text{VerifyCred}(\text{cred}, \phi, \text{tg}, \mathsf{T}_{\text{App}})$: This algorithm is run by any entity who wishes to verify if a credential embedding private attributes satisfies the application-specific statement ϕ and that the user showing it is the rightful owner. On input a credential cred and tag tg , it outputs bit $b = 1$ if it accepts the credential as valid and $b = 0$ otherwise. Also, it records tg in its database of redeemed tags T_{App} .

Properties. An S3ID scheme requires to provide unforgeability, Sybil-resistance, and strong unlinkability. Similar to the tACT security definitions, unforgeability and Sybil-resistance are proven in the presence of an adversary $\mathcal{A}$ who can corrupt up to $t - 1$ out of N issuers. Additionally, this adversary holds the ability to corrupt any number of users and applications within the system. For strong unlinkability, $\mathcal{A}$ presents the strongest category of adversaries who can corrupt *any* number of issuers and verifiers.

Unforgeability. This property captures the infeasibility of an adversarial user to generate credentials to prove a statement ϕ , where ϕ cannot be satisfied by any subset of attributes for which the adversary has obtained sufficiently many valid partial micro-credentials (including corrupted issuers). In interactive algorithms such as Dedup and MicroCred, we use subscript U and I to distinguish interactions on the user side and the issuer side, respectively.

Definition 6 (Unforgeability). A S3ID scheme is unforgeable if for any PPT adversary $\mathcal{A}$ and any $T \geq 0$

$$\text{Adv}_{\text{S3ID}, \mathcal{A}}^{\text{UF}}(\lambda) := \Pr[\text{UF}_{\text{S3ID}, \mathcal{A}}(\lambda) \rightarrow 1] \leq \text{negl}(\lambda),$$

where $\text{UF}_{\text{S3ID}, \mathcal{A}}(\lambda)$ is defined in [Figure 6](#).

Sybil-Resistance. This definition addresses the impossibility of an adversary to obtain more tokens than the number of users it controls during the deduplication mechanism, and therefore breaking the uniqueness of issued user identities. We refer to [Figure 7](#) for the detailed game setup, where the adversary begins with initializing R users, each possessing a unique deduplication attribute. Subsequently, the adversary has the ability to request tokens for the users it controls. The adversary succeeds by outputting a database $\mathsf{T}_{\text{Dedup}}$ including more than R valid tokens.

Definition 7 (Sybil-resistance). A S3ID system is Sybil-resistant if for any PPT adversary $\mathcal{A}$ and any $R \geq 0$

$$\text{Adv}_{\text{S3ID}, \mathcal{A}, R}^{\text{SR}}(\lambda) := \Pr[\text{SR}_{\text{S3ID}, \mathcal{A}, R}(\lambda) \rightarrow 1] \leq \text{negl}(\lambda),$$

Game $\text{UF}_{\text{S3ID}, \mathcal{A}}(\lambda)$	$\mathcal{O}_{\text{MicroCred}}(\text{pp})$
$(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda)$ $\text{flag} = 0, R = 0, T = 0, \text{T}_{\text{Dedup}} = \emptyset$ $\mathcal{Q} = \emptyset, \text{CS} = \emptyset, \text{HS} := [1, N] \setminus \text{CS}$ $(\text{CS}, \mathcal{Q}, \text{cred}, \phi) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Dedup}}, \mathcal{O}_{\text{MicroCred}}, \mathcal{O}_{\text{Corrupt}}}(\text{pp})$ $\forall k \in R: \mathbb{A}_k := \{a_i \mid (k, i, *, a_i) \in \mathcal{Q} \wedge T_{ik} + \text{CS} \geq t\}$ return $ \text{CS} < t \wedge \forall k \in R, \forall \mathbb{A}' \subset \mathbb{A}_k: \phi(\mathbb{A}') = 0 \wedge \text{VerifyCred}(\text{cred}, \phi, *, *) = 1$	Assert $(j \in \text{HS})$ $\sigma_{ij} \leftarrow (\text{MicroCred}_{\text{U}}(a_i, \text{prv}_{\text{U}}[k]), \text{MicroCred}_{\text{I}}(\text{prv}_j, \text{T}_{\text{Dedup}}))$ $\{\text{partial credential on } a_i\}$ $T_{ik} = T_{ik} + 1$ $\mathcal{Q} = \mathcal{Q} \cup (k, i, \text{prv}_{\text{U}}[k], a_i)$ $\text{flag} = 1$ return $\top$
$\mathcal{O}_{\text{Dedup}}(\text{pp})$ $(\text{pp}_{\text{U}}, \text{prv}_{\text{U}}) \leftarrow R = R + 1$ $(\text{Dedup}_{\text{U}}(a), \text{Dedup}_{\text{I}}(\text{prv}_j, \text{T}_{\text{Dedup}}))$ if $\text{pp}_{\text{U}} = \perp$ then abort else $\text{T}_{\text{Dedup}} = \text{T}_{\text{Dedup}} \cup \text{pp}_{\text{U}}$ return $\top$	if $\text{flag} = 1$ or $k \in \text{CS}$ then abort else $\text{CS} = \text{CS} \cup \{k\} \wedge \text{HS} = \text{HS} \setminus \{k\}$ return prv_k

Fig. 6: Unforgeability experiment for S3ID.

where $\text{SR}_{\text{S3ID}, \mathcal{A}, R}(\lambda)$ is defined in [Figure 7](#).

Game $\text{SR}_{\text{S3ID}, \mathcal{A}, R}(\lambda)$	$\mathcal{O}_{\text{Dedup}}(\text{pp})$
$(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{Setup}(1^\lambda)$ $R = 0, \text{CS} = \emptyset$ $\text{T}_{\text{Dedup}} = \emptyset, \text{T}_{\text{Dedup}} = S$ $\text{CS}, \text{T}_{\text{Dedup}}, \{\pi_i, \text{bR}_i\}_{i \in [S]} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Dedup}}, \mathcal{O}_{\text{Corrupt}}}(\text{pp})$ return $ \text{CS} < t \wedge S > R \wedge (\forall i \in [S] \wedge \text{token}_i \in \text{T}_{\text{Dedup}}, \text{tACT.Verify}(\text{token}_i, \pi_i, \text{bR}_i) = 1)$	$R = R + 1$ $(\text{pp}_{\text{U}}, \text{prv}_{\text{U}}) \leftarrow (\text{Dedup}_{\text{U}}(a), \text{Dedup}_{\text{I}}(\text{prv}_j, \text{T}_{\text{Dedup}}))$ if $\text{pp}_{\text{U}} = \perp$ then abort else $\text{T}_{\text{Dedup}} = \text{T}_{\text{Dedup}} \cup \text{pp}_{\text{U}}$ return T_{Dedup}
	$\mathcal{O}_{\text{Corrupt}}(k)$ if $k \in \text{CS}$ then abort else $\text{CS} = \text{CS} \cup \{k\} \wedge \text{HS} = \text{HS} \setminus \{k\}$ return prv_k

Fig. 7: Sybil resistance experiment for S3ID.

Strong Unlinkability. It should be infeasible for an adversarial verifier potentially colluding with any number of issuers and verifiers to link the execution of AppCred with either another execution of AppCred (unlinkability across applications), or with the executions of MicroCred and Dedup algorithms

for a specific set of attributes. The $\text{SUNLINK}_{\text{S3ID}, \mathcal{A}}(\lambda)$ game is defined in [Figure 8](#). The adversary $\mathcal{A}$ can instruct users in the system with a specific set of attributes to generate tokens and micro-credentials $\{\mathbf{s}_i\}_{i=1}^L$ respectively through the $\mathcal{O}_{\text{Dedup}}$ and $\mathcal{O}_{\text{MicroCred}}$ oracles. Then $\mathcal{A}$ chooses two distinct sets $(\text{token}_b, \{\mathbf{s}_{i,b}\}_{i=1}^L), b \in \{0, 1\}$, among which one of them is randomly picked to generate cred , through the $\mathcal{O}_{\text{AppCred}}$ oracle. The adversary should distinguish cred for two different client/attribute pairs; or it needs to distinguish cred for the same client but two different AppCred sessions. The definition requires that VerifyCred succeeds on both the credentials generated based on requests provided by the adversary.

Definition 8 (Strong Unlinkability). A S3ID system achieves Strong Unlinkability if for PPT adversary $\mathcal{A}$ the advantage defined as

$$\text{Adv}_{\text{S3ID}, \mathcal{A}}^{\text{SUNLINK}}(\lambda) := |2 \Pr[\text{SUNLINK}_{\text{S3ID}, \mathcal{A}}(\lambda) \rightarrow 1] - 1|,$$

where $\text{SUNLINK}_{\text{S3ID}, \mathcal{A}}(\lambda)$ is defined in [Figure 8](#), is negligible, i.e.,

$$\text{Adv}_{\text{S3ID}, \mathcal{A}}^{\text{SUNLINK}}(\lambda) \leq \text{negl}(\lambda).$$

Game $\text{SUNLINK}_{\text{S3ID}, \mathcal{A}}(\lambda)$

$(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]}) \leftarrow \text{S3ID.Setup}(1^\lambda)$
 $b \xleftarrow{\$} \{0, 1\}$
 $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Dedup}}, \mathcal{O}_{\text{MicroCred}}, \mathcal{O}_{\text{AppCred}}}(\text{pp}, \{\text{prv}_j\}_{j \in [1, N]})$
return $(b' == b)$

$\mathcal{O}_{\text{Dedup}}(\text{pp})$
 $(\text{pp}_U, \text{prv}_U) \leftarrow (\text{Dedup}_U(a), \text{Dedup}_I(\text{prv}_j, \text{T}_{\text{Dedup}}))$
if $\text{pp}_U = \perp$ **then abort**
else $\text{T}_{\text{Dedup}} = \text{T}_{\text{Dedup}} \cup \text{pp}_U$
return $\top$

$\mathcal{O}_{\text{MicroCred}}(\text{pp})$
 $s_i \leftarrow (\text{MicroCred}_U(a_i, \text{prv}_U), \text{MicroCred}_I(\{\text{prv}_j\}_{j \in [1, N]}, \text{T}_{\text{Dedup}}))$
if $s_i = \perp$ **then abort**
else $\mathcal{Q} = \mathcal{Q} \cup (i, s_i)$
return $\top$

$\mathcal{O}_{\text{AppCred}}(\text{pp}, \text{pp}_{U_0}, \text{pp}_{U_1}, l_0, l_1, \phi_0, \phi_1)$
if $\phi_{0,1}(\{a_{i,0}\}_{i=1}^L) = 0$ or $\phi_{0,1}(\{a_{i,1}\}_{i=1}^L) = 0$ **then abort**
 $(\text{cred}_0, \text{tag}_0) \leftarrow \text{AppCred}(\{a_{i,b}\}_{i=1}^L, \{s_{i,b}\}_{i=1}^L, \text{prv}_{U_b}, l_0, \phi_0)$
 $(\text{cred}_1, \text{tag}_1) \leftarrow \text{AppCred}(\{a_{i,0}\}_{i=1}^L, \{s_{i,0}\}_{i=1}^L, \text{prv}_{U_0}, l_b, \phi_b)$
if $\text{VerifyCred}(\text{cred}_0, \phi_0, \text{tag}_0) = 0$ or
 $\text{VerifyCred}(\text{cred}_1, \phi_b, \text{tag}_1) = 0$ **then abort**
return $(\text{cred}_b, \text{tag}_b)$

Fig. 8: Strong unlinkability experiment for S3ID.

Besides the formal security properties, we also want the following properties to hold:

Non-Transferability. To prevent the transfer of credentials among users, it needs to be ensured that issued credentials can only be released by the legitimate owner and cannot be transferred to any other individual.

Non-Interactiveness. After establishing consensus on system parameters in the key distribution and setup phase, the issuers can function independently without the need to further synchronize or coordinate. Additionally, the system enables users to generate unique application-specific credentials without the necessity of interacting with the issuers for each transaction.

3.2 S3ID Design

Now we present an application of our tACT system by employing it as a fundamental building block for our S3ID scheme. This is detailed in [Algorithm 2](#). In the S3ID.Dedup algorithm, a token generated in the underlying tACT algorithm serves as a distinctive high-entropy function derived from the unique attribute $\mathbf{a}$, enabling user deduplication. Upon successful verification of token, the issuers proceed by comparing token against their database of redeemed tokens and reject if the value already occurs there. Otherwise, they update the database by adding the redeemed token to $\mathbf{T}_{\text{Dedup}}$. In $\mathbf{T}_{\text{Dedup}}$, alongside the redeemed token, issuers also keep a commitment cm_k to the unique key k registered by user, and a binary vector $\mathbb{I}\mathbb{D}\mathbb{X}$ of length L initially set to all zero, where $\mathbb{I}\mathbb{D}\mathbb{X}[i] = 1$ shows the user is issued a micro-credential for her attribute $\mathbf{a}_i$ (see S3ID.MicroCred for more details).

In the S3ID.MicroCred algorithm, the user, to acquire anonymous micro-credentials for its desired attribute $\mathbf{a}_i$, generates a commitment cm_i on $(k, \mathbf{a}_i)$, and a proof of knowledge π_i to prove the consistency of cm_i with the commitment cm_k on the registered key. Then she submits $\{\text{token}, \text{cm}_i, \pi_i\}$ to the issuers. Issuer I_j checks the latest version of deduplication table $\mathbf{T}_{\text{Dedup}}$. If $\text{token} \in \mathbf{T}_{\text{Dedup}}$ and $\mathbb{I}\mathbb{D}\mathbb{X}[i] = 0$ and the proof π_i passes the verification, it generates a partial tSW_{Ped} signature on commitment cm_i to $\mathbf{a}_i$ bound to user-unique key k . The user then proceeds to gather sufficient partial signatures, combines them to form a full signature, and converts the result to a tSW signature subsequently. The resulting s_i is a micro-credential on attribute $\mathbf{a}_i$ which is bound to key k .

In the S3ID.AppCred algorithm, the user is empowered to generate application-specific credentials. We assume a unique message l for each application that specifies the application name (e.g., “participate in auction X”). Also consider $\mathbb{A}_l = \{\mathbf{a}_i\}_{i \in \mathbb{Q}}, \mathbb{Q} \subset [1, L]$ as the subset of attributes corresponding to the application ($\mathbb{A}_l \subset \mathbb{A}$) and ϕ as the application-specific statement (or predicate). To get an application-specific credential for an application with message l , the user locally aggregates the relevant micro-credentials encapsulating attributes in $\mathbb{A}_l$

with respect to the message l , and generates one small application credential ζ , which are the aggregated signatures masked by pk^r for uniformly random r . The user also generates τ as a Pedersen commitment to the designated attributes together with the registered key k using randomness r . The presence of k in τ enforces users to build application credentials based on their genuinely owned micro-credentials, reinforcing non-transferability.

Furthermore, micro-credentials are bound to unique attribute indices used by the verifier in the verification equation. This safeguards against replace attacks and prevents users from acquiring application credentials by substituting a different combination of micro-credentials with encoded attributes that do not belong to $\mathbb{A}_l$. To prevent double submissions and achieve Sybil-resistance, the user generates a tag tg as a PRF evaluated under the registered key k on the unique application message l . Therefore, tg is unique per user and per application. The PRF should have algebraic properties such that they can be easily be proven using efficient ZKPoK, i.e., Schnorr proofs in our case. Two possible instantiations of such PRFs are the Naor-Pinkas-Reingold PRF [NPR99] $\text{PRF}(k, l) = H(l)^k$ or Dodis-Yampolskiy PRF [DY05] $\text{PRF}(k, l) = g^{(k+l)^{-1}}$, where for our instantiation we choose the former. The user then computes a ZKPoK proof π to demonstrate the correct evaluation of tg on the message l with respect to the key k encapsulated in all of the user's micro-credentials. Additionally, this proof serves to confirm that the user claiming these attributes is indeed the rightful owner. Moreover, the user proves the validity of the masked tSW signatures ζ without revealing it, which amounts to the pairing equation $e(\zeta, g) = e(\prod_{i \in \mathbb{Q}} H(i) \cdot \tau, \text{pk})$ and can be efficiently proven using a Groth-Sahai ZKPoK proof [GS12] (see Section 4 for a more detailed discussion about the proven statement). Finally, the user sets $\text{cred} = (\tau, \pi, \text{tg})$ as its self-sovereign application-specific credential and sends it to the relying party.

In S3ID.VerifyCred algorithm, any party who knows public parameters pp can verify a user's credential cred . The verifier first checks if the tag tg does not occur in the unique per application database T_{App} of submitted tags, and subsequently verifies the proof π . If the verification succeeds, the verifier is simultaneously convinced that all the micro-credentials contained within the application credential have been appropriately signed by a threshold of issuers.

Remark. We note that establishing a one-to-one correspondence between the user and the claimed attributes can be achieved through the utilization of privacy-preserving oracles [ZCC⁺16, ZMM⁺20]. In this context, our systems can seamlessly integrate with a legacy-compatible oracle system. This integration enables users to provide an additional oracle proof, denoted as $\pi^{\mathcal{O}}$, thus substantiating ownership of the attribute a concealed within the commitment cm , ultimately demonstrating unique personhood while maintaining privacy.

For the security analysis of S3ID, we refer to Appendix C.3.

Algorithm 2 S3ID Construction

- 1: **S3ID.Setup**($1^\lambda, t, N$)
- 2: Let $\mathbb{A} = \{a_1, \dots, a_L\}$ denote the set of admissible attributes and $\mathbb{I} = \{I_1, \dots, I_N\}$ denote the set of authorized issuers. {unique attribute a is defined at index 0.}
- 3: Run $(pp', \{prv_j\}_{j=1}^N) \leftarrow \text{tACT.Setup}(1^\lambda, t, N)$ and $(pp'' = \{g, u_0, u_1, \dots, u_L\}) \leftarrow \text{Ped.Setup}(1^\lambda, L + 1)$. Let $H'' : \{0, 1\}^* \rightarrow \mathbb{F}_p$ and $h : \mathbb{G}_1 \rightarrow \mathbb{F}_p$ be secure hash functions.
- 4: Issuers initialize a shared table $T_{\text{Dedup}} := \emptyset$
- 5: **Output:** System public parameters $pp = (pp', pp'', H'', h, T_{\text{Dedup}})$, issuers' private parameter $\{prv_j\}_{j \in [1, N]}$
- 6: **S3ID.Dedup**($a, \{prv_j\}_{j \in [1, N]}, T_{\text{Dedup}}$)
- 7: The user on input private unique attribute a and issuers on input $\{prv_j\}_{j \in [1, N]}$ interact in the following way:
 - a: User runs $(st_{Rg}, cm) \leftarrow \text{tACT.Register}(a), (bR, rand_a) \leftarrow \text{tACT.TokenRequest}(st_{Rg}, cm)$
 - b: I_j runs $\{bT_{jk}\}_{k \in [1, n]} \leftarrow \text{tACT.tlssue}(bR, prv_j)$
 - c: User runs $token, \{s_k\}_{k \in [1, n]} \leftarrow \text{tACT.}(Aggregate, Unblind)(\{bT_{jk}\}_{j \in [t]}, rand_a)$
 - d: User runs $\pi \leftarrow \text{tACT.Prove}(token, \{s_k\}_{k \in [1, n]}, rand_a)$
 - e: User samples a key $k \leftarrow \mathbb{F}_p$ and outputs $cm_k = \text{Ped.Commit}(k; r_k)$, where $r_k \leftarrow \mathbb{F}_p$
 - f: Issuers run $bit \leftarrow \text{tACT.Verify}(token, \pi, bR)$
 - g: If $(bit = 0 \vee (bit = 1 \wedge token \in T_{\text{Dedup}}))$, abort.
 - h: Initialize binary vector $\mathbb{ID}\mathbb{X}$ of length L with all zeros.
 - i: Set $T_{\text{Dedup}} \leftarrow T_{\text{Dedup}} \cup \{token, cm_k, \mathbb{ID}\mathbb{X}\}$.
- 8: **Output:** $pp_U \leftarrow \{token, cm_k, \mathbb{ID}\mathbb{X}\}$, $prv_U \leftarrow \{a, k\}$, T_{Dedup}
- 9: **S3ID.MicroCred**($\{a_i\}_{i=1}^L, pp_U, prv_U, \{prv_j\}_{j \in [1, N]}, T_{\text{Dedup}}$)
- 10: The user on input prv_U and private attribute a_i for which she seeks to obtain a micro-credential, and issuers on input $\{prv_j\}_{j \in [1, N]}$ interact in the following way:
 - a: User picks random $r_i \leftarrow \mathbb{F}_p$ and computes $cm_i = \text{Ped.Commit}(k, a_i; r_i) = g^{r_i} \cdot u_0^k \cdot u_i^{a_i}$.
 - b: Generates $\pi_i = \text{ZKPoK}\{(a_i, k, r_i, r_k) : cm_i = \text{Ped.Commit}(k, a_i; r_i) \wedge cm_k = \text{Ped.Commit}(k; r_k) \wedge \phi(a_i) = 1\}$
 - c: Outputs $request_i \leftarrow \{token, cm_i, \pi_i\}$
 - d: I_j parses $request_i = \{token, cm_i, \pi_i\}$
 - e: If $(token \in T_{\text{Dedup}} \wedge \mathbb{ID}\mathbb{X}[i] = 0)$, go to next step. Else, abort.
 - f: If $\text{ZKVerify}((cm_i, cm_k, \phi), \pi_i) = 0$, abort.
 - g: Signs $\sigma_{ij} = \text{tSW}_{\text{Ped}}.\text{Sign}(prv_j, cm_i, i)$ and sets $\mathbb{ID}\mathbb{X}[i] \leftarrow 1$
 - h: User computes full signature $\sigma_i = \text{tSW}.\text{Comb}(\{\sigma_{ij}\}_{j \in [t]})$
 - i: Unblinds cm_i by computing $s_i = \sigma_i \cdot pk^{-r_i}$
- 11: **Output:** s_i, T_{Dedup}
- 12: **S3ID.AppCred**($\{a_i\}_{i=1}^L, \{s_i\}_{i=1}^L, prv_U, l, \phi, \mathbb{A}_l$)
- 13: The application relying party initializes a table $T_{\text{App}} := \emptyset$
- 14: The user, on input unique application message l , application-specific statement ϕ , and $\mathbb{A}_l = \{a_i\}_{i \in \mathbb{Q}}, \mathbb{Q} \subset [1, L]$ as the subset of attributes corresponding to the application, follows these steps:
 - a: Picks $r \leftarrow \mathbb{F}_p$ and calculates $\tau = g^r \cdot u_0^{|\mathbb{Q}|k} \cdot \prod_{i \in \mathbb{Q}} u_i^{a_i}$
 - b: Generates $\zeta = pk^r \cdot \prod_{i \in \mathbb{Q}} s_i$
 - c: Sets $tg = \text{PRF}(k, l)$
 - d: Generates $\pi = \text{ZKPoK}\{(\mathbb{A}_l, k, \zeta, r) : \tau = g^r \cdot u_0^{|\mathbb{Q}|k} \cdot \prod_{i \in \mathbb{Q}} u_i^{a_i} \wedge tg = \text{PRF}(k, l) \wedge \phi(\mathbb{A}_l) = 1 \wedge e(\zeta, g) = e(\prod_{i \in \mathbb{Q}} H(i) \cdot \tau, pk)\}$
- 15: **Output:** $cred \leftarrow \{\tau, \pi, tg$

```

16: S3ID.VerifyCred(cred, l, φ, tg, TApp)
17: The verifying party does the following:
18: Parses cred = {τ, π}
19: If all of the following conditions are met, set bit ← 1. Otherwise, set bit ← 0.
    a: Tag tg ∉ TApp {TApp = database of redeemed tags.}
    b: ZKVerify((τ, tg, l, φ), π) = 1
20: Output: bit, TApp ← TApp ∪ tg

```

4 Evaluation and Experimental Results

We implemented tACT from Section 2 and S3ID¹⁰ from Section 3.2 in Rust using the ark-bls12-381¹¹ and groth-sahai-rs¹² crate for implementations of the pairing-friendly BLS12-381 curve and the Groth-Sahai proof system. As this curve provides a Type-3 pairing, we applied a generic compiler [AGH15, AHO16] at the cost of duplicating operations and values in the second source group. With the implementation, we evaluated the runtime costs of the individual algorithms. As discussed in Section 2.3, the parameters were set to $t = \frac{N}{2} + 1$ and $t' = \frac{n}{2} + 1$. For the number of issuers, N , we considered values of 4 (as in the evaluation of CanDID) and 64 and for n we compared choices of 30, 40, and 128.

Table 2: tACT runtime in milliseconds with $t' = \frac{n}{2} + 1$.

$N = 4, t = \frac{N}{2} + 1$	$n = 30$	$n = 40$	$n = 128$
Register	0.99	1.13	1.10
TokenRequest	251.7	444.6	4424
tlssue	25.9	41.0	138.5
(Aggregate, Unblind)	14.6	24.9	83.4
Prove	31.0	45.2	130.8
Verify	168.4	325.3	2756
$N = 64, t = \frac{N}{2} + 1$	$n = 30$	$n = 40$	$n = 128$
Register	1.16	1.17	1.17
TokenRequest	279.9	438.6	4318
tlssue	27.9	38.2	128.2
(Aggregate, Unblind)	88.6	116.3	375.9
Prove	33.8	42.6	211.5
Verify	200.8	326.7	2906

The benchmarks were performed with an Intel Core i7-1265U and 16 GB of RAM. The runtime results of tACT are depicted in Table 2. We observe that the

¹⁰ See <https://github.com/ait-crypto/s3id/> for the code.

¹¹ https://crates.io/crates/ark-bls12_381, version 0.5.0.

¹² <https://github.com/jdwhite48/groth-sahai-rs>

biggest impact on the performance is the choice of t' and n . This observation matches the intuition based on the construction where the **TokenRequest**, **tlssue**, (**Aggregate**, **Unblind**), **Prove** and **Verify** linearly depend on t' or n . Notably, only (**Aggregate**, **Unblind**) also depend linearly on t whereas **tlssue** is executed in parallel at the N issuers.

For **S3ID**, the benchmark results are presented in [Table 3](#). We observe that for algorithms **Dedup** and **MicroCred** the runtime reflects the fact that both depend on the number of issuers and the security parameter. Notably, for a larger security parameter and more issuers, the runtime of **Dedup** is ≈ 10 seconds at the same magnitude as **CanDID** [MMZ⁺21] while the latter uses a significantly smaller security parameter and only 4 issuers.

Also note that both **AppCred** and **VerifyCred** are independent of the choices of the parameters and the variations are due to noise on the benchmarking system. The implementation currently does not include the evaluation of ϕ as the policies depend on the application domain. Concrete policies and the representation of ϕ will have a non-negligible impact on the performance. Hence, the runtimes **AppCred** and **VerifyCred** are lower bounds. However, note that the runtime of **MicroCred** is linear in the number of attributes L whereas **AppCred** and **VerifyCred** is linear in the size of $\mathbb{A}_l$ instead.

In the implementation of **AppCred** and **VerifyCred** we also split the NIZK proof into a Groth-Sahai proof of the pairing equation $e(\zeta, g) = e(\prod_{i \in \mathbb{Q}} \mathbf{H}(i) \cdot \tau, \mathbf{pk})$ and a Schnorr proof of the other statements – the well-formedness of $\tau = g^r u_0^{|\mathbb{Q}|^k} \prod_{i \in \mathbb{Q}} u_i^{a_i}$ and the evaluation of the PRF $\mathbf{tg} = \text{PRF}(k, l) = \mathbf{H}(l)^k$. To bind the two proofs together and to ensure the non-malleability of the Groth-Sahai proof, the Groth-Sahai proof is included in the hash of the Fiat-Shamir transform of the Schnorr proof.

Table 3: **S3ID** runtime in milliseconds with $t' = \frac{n}{2} + 1$ and $L = 16$ and $|\mathbb{A}_l| = \frac{L}{2}$.

$N = 4, t = \frac{N}{2} + 1$ $n = 30$ $n = 40$ $n = 128$			
Dedup	629.8	968.1	8011
MicroCred	136.5	138.4	134.8
AppCred	27.3	27.5	27.4
VerifyCred	30.5	30.7	30.5
$N = 64, t = \frac{N}{2} + 1$ $n = 30$ $n = 40$ $n = 128$			
Dedup	1235	1779	10754
MicroCred	1645	1649	1641
AppCred	28.8	27.5	27.6
VerifyCred	30.8	30.3	30.3

Furthermore, while we observe an increase in the runtimes of **Dedup** and **MicroCred** as the number of issuers grows, the protocol can be scaled to ac-

commodate more issuers without significantly impacting overall runtime. This is because these algorithms are parallelizable, single-round, and require only one-time execution per user. The increase in the benchmarks is caused by running all issuers on a system with a higher number of issuers than number of cores.

5 Conclusion

This paper first introduces tACT, marking a shift in anonymous counting tokens by operating in a distributed environment. Utilizing tACT as a core element, we present a system called S3ID that achieves distributed threshold issuance, unlinkable multi-show selective disclosure, non-interactive, and non-transferable credentials. We provide an implementation of our system along with benchmarks which show that our work addresses the shortcomings of current self-sovereign identity systems.

Acknowledgments. We would like to thank the anonymous reviewers for their valuable comments. This work received funding from the EU research project SWARMCHESTRATE (project no. 101135012) and the European Union’s Horizon Europe project SUNRISE (project no. 101073821), and by PREPARED, a project funded by the Austrian security research programme KIRAS of the Federal Ministry of Finance (BMF).

References

- AGH15. Joseph A. Akinyele, Christina Garman, and Susan Hohenberger. Automating fast and secure translations from type-i to type-iii pairing schemes. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1370–1381. ACM, 2015.
- AGOT14. Masayuki Abe, Jens Groth, Miyako Ohkubo, and Takeya Tango. Converting cryptographic schemes from symmetric to asymmetric bilinear groups. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 241–260. Springer, 2014.
- AHO16. Masayuki Abe, Fumitaka Hoshino, and Miyako Ohkubo. Design in type-i, run in type-iii: Fast and scalable bilinear-type conversion using integer programming. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 387–415. Springer, 2016.
- BBG⁺20. James Henry Bell, Kallista A Bonawitz, Adrià Gascón, Tancrede Lepoint, and Mariana Raykova. Secure single-server aggregation with (poly) logarithmic overhead. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1253–1269, 2020.

- BCC⁺21. Lorenz Breidenbach, Christian Cachin, Benedict Chan, Alex Coventry, Steve Ellis, Ari Juels, Farinaz Koushanfar, Andrew Miller, Brendan Magauran, Daniel Moroz, et al. Chainlink 2.0: Next steps in the evolution of decentralized oracle networks. *Chainlink Labs*, 1:1–136, 2021.
- BCJ⁺24. Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindstrøm, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zklogin: Privacy-preserving blockchain authentication with existing credentials. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3182–3196, 2024.
- BCK⁺22. Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 517–550. Springer, 2022.
- BIK⁺17. Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.
- BKKJ⁺17. Maria Borge, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, and Bryan Ford. Proof-of-personhood: Redemocratizing permissionless cryptocurrencies. In *2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 23–26. IEEE, 2017.
- BL22. Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 193–207. ACM, 2022.
- BLS01. Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *Advances in Cryptology- ASIACRYPT 2001*, volume 2248, pages 514–532. Springer, 2001.
- BRS23. Fabrice Benhamouda, Mariana Raykova, and Karn Seth. Anonymous counting tokens. In *ASIACRYPT*, 2023.
- C⁺16. Christian Cachin et al. Architecture of the hyperledger blockchain fabric. In *Workshop on distributed cryptocurrencies and consensus ledgers*, volume 310, pages 1–4. Chicago, IL, 2016.
- CCK⁺23. Panagiotis Chatzigiannis, Konstantinos Chalkias, Aniket Kate, Easwar Vivek Mangipudi, Mohsen Minaei, and Mainack Mondal. Sok: Web3 recovery mechanisms. *Cryptology ePrint Archive*, 2023.
- CDL⁺13. Jan Camenisch, Maria Dubovitskaya, Anja Lehmann, Gregory Neven, Christian Paquin, and Franz-Stefan Preiss. Concepts and languages for privacy-preserving attribute-based authentication. In *Policies and Research in Identity Management: Third IFIP WG 11.6 Working Conference, IDMAN 2013, London, UK, April 8-9, 2013. Proceedings 3*, pages 34–52. Springer, 2013.
- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *Annual International Cryptology Conference*, pages 174–187. Springer, 1994.

- CKM23. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 678–709. Springer, 2023.
- CKS24. Elizabeth Crites, Aggelos Kiayias, and Amirreza Sarencheh. Syra: Sybil-resilient anonymous signatures with applications to decentralized identity. *Cryptology ePrint Archive*, 2024.
- CL04. Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In *Advances in Cryptology-CRYPTO 2004: 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004. Proceedings 24*, pages 56–72. Springer, 2004.
- CLPK22. Aisling Connolly, Pascal Lafourcade, and Octavio Perez Kempner. Improved constructions of anonymous credentials from structure-preserving signatures on equivalence classes. In *Public-Key Cryptography-PKC 2022: 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Virtual Event, March 8–11, 2022, Proceedings, Part I*, pages 409–438. Springer, 2022.
- Cra96. Ronald Cramer. Modular design of secure yet practical cryptographic protocols. *Ph. D.-thesis, CWI and U. of Amsterdam*, 2, 1996.
- CS97. Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. *Technical Report/ETH Zurich*, 260, 1997.
- DGS⁺18. Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. Privacy pass: Bypassing internet challenges anonymously. *Proc. Priv. Enhancing Technol.*, 2018(3):164–180, 2018.
- dif20. Decentralized Identity Foundation. <https://identity.foundation/>, 2020.
- DKL⁺23. Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKyah Tyner. Threshold BBS+ signatures for distributed anonymous credential issuance. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 773–789. IEEE, 2023.
- DY05. Yevgeniy Dodis and Aleksandr Yampolskiy. A verifiable random function with short proofs and keys. In *Public Key Cryptography-PKC 2005: 8th International Workshop on Theory and Practice in Public Key Cryptography, Les Diablerets, Switzerland, January 23-26, 2005. Proceedings 8*, pages 416–431. Springer, 2005.
- fac18. Facebook data breach. <https://www.nytimes.com/2018/09/28/technology/facebook-hack-data-breach.html>, 2018.
- FG18. Georg Fuchsbauer and Romain Gay. Weakly secure equivalence-class signatures from standard assumptions. In *Public-Key Cryptography-PKC 2018: 21st IACR International Conference on Practice and Theory of Public-Key Cryptography, Rio de Janeiro, Brazil, March 25-29, 2018, Proceedings, Part II 21*, pages 153–183. Springer, 2018.
- FHS19. Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 32:498–546, 2019.
- FS86. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- Gol09. Oded Goldreich. *Foundations of cryptography: volume 2, basic applications*. Cambridge university press, 2009.

- GS12. Jens Groth and Amit Sahai. Efficient noninteractive proof systems for bilinear groups. *SIAM J. Comput.*, 41(5):1193–1232, 2012.
- HIJ⁺21. Sharon Huang, Subodh Iyengar, Sundar Jeyaraman, Shiv Kushwah, Chen-Kuei Lee, Zutian Luo, Payman Mohassel, Ananth Raghunathan, Shaahid Shaikh, Yen-Chieh Sung, et al. Dit: De-identified authenticated telemetry at scale, 2021.
- HP23. Chloé Héban and David Pointcheval. Traceable constant-size multi-authority credentials. *Information and Computation*, page 105060, 2023.
- HS21. Lucjan Hanzlik and Daniel Slamanig. With a little help from my friends: Constructing practical anonymous credentials. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2004–2023, 2021.
- KHG12. Aniket Kate, Yizhou Huang, and Ian Goldberg. Distributed key generation in the wild. *Cryptology ePrint Archive*, 2012.
- KLOR20. Ben Kreuter, Tancrede Lepoint, Michele Orrù, and Mariana Raykova. Anonymous tokens with private metadata bit. In *Advances in Cryptology—CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I 40*, pages 308–336. Springer, 2020.
- KRB⁺24. Ioanna Karantaidou, Omar Renawi, Foteini Baldimtsi, Nikolaos Kamari-nakis, Jonathan Katz, and Julian Loss. Blind multisignatures for anonymous tokens with decentralized issuance. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1508–1522, 2024.
- MBG⁺23. Omid Mir, Balthazar Bauer, Scott Griffy, Anna Lysyanskaya, and Daniel Slamanig. Aggregate signatures with versatile randomization and issuer-hiding multi-authority anonymous credentials. In *CCS*, pages 30–44. ACM, 2023.
- MMZ⁺21. Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. Candid: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1348–1366. IEEE, 2021.
- ms23. Microsoft data breach. <https://www.reuters.com/world/us/chinese-hackers-stole-60000-emails-us-state-department-microsoft-hack-senate-2023-09-27/>, 2023.
- MSM23. Omid Mir, Daniel Slamanig, and René Mayrhofer. Threshold delegatable anonymous credentials with controlled and fine-grained delegation. *IEEE Transactions on Dependable and Secure Computing*, pages 1–16, 2023.
- NPR99. Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and kdcs. In Jacques Stern, editor, *Advances in Cryptology - EUROCRYPT '99*, volume 1592, pages 327–346. Springer, 1999.
- Ped91. Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
- PS16. David Pointcheval and Olivier Sanders. Short randomizable signatures. In *Topics in Cryptology-CT-RSA 2016: The Cryptographers’ Track at the RSA Conference 2016, San Francisco, CA, USA, February 29-March 4, 2016, Proceedings*, pages 111–126. Springer, 2016.

- SABB⁺18. Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. *arXiv preprint arXiv:1802.07344*, 2018.
- San20. Olivier Sanders. Efficient redactable signature and application to anonymous credentials. In *Public-Key Cryptography–PKC 2020: 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4–7, 2020, Proceedings, Part II*, pages 628–656. Springer, 2020.
- SKP⁺23. Tanusree Sharma, Yujin Kwon, Kornrapat Pongmala, Henry Wang, Andrew Miller, Dawn Song, and Yang Wang. Unpacking how decentralized autonomous organizations (daos) work in practice. *arXiv preprint arXiv:2304.09822*, 2023.
- Sov18. Sovrin. Public service utility enabling self-sovereign identity on the internet. <https://sovrin.org/>, 2018.
- SW13. Hovav Shacham and Brent Waters. Compact proofs of retrievability. *Journal of cryptology*, 26(3):442–483, 2013.
- TBM⁺20. Sri Aravinda Krishnan Thyagarajan, Adithya Bhat, Giulio Malavolta, Nico Döttling, Aniket Kate, and Dominique Schröder. Verifiable timed signatures made practical. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1733–1750, 2020.
- W3C18. W3C. Decentralized identifiers (DIDs). v0.11: data model and syntaxes for decentralized identifiers. <https://w3c-ccg.github.io/did-spec/>, 2018.
- WGW⁺23. Ke Wang, Jianbo Gao, Qiao Wang, Jiashuo Zhang, Yue Li, Zhi Guan, and Zhong Chen. Hades: Practical decentralized identity with full accountability and fine-grained sybil-resistance. In *Proceedings of the 39th Annual Computer Security Applications Conference*, pages 216–228, 2023.
- yah17. Yahoo data breach. <https://www.nytimes.com/2017/10/03/technology/yahoo-hack-3-billion-users.html>, 2017.
- ZCC⁺16. Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. Town crier: An authenticated data feed for smart contracts. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 270–282, 2016.
- ZMM⁺20. Fan Zhang, Deepak Maram, Harjasleen Malvai, Steven Goldfeder, and Ari Juels. Deco: Liberating web data using decentralized oracles for tls. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1919–1938, 2020.

A Related Work

Subsequently, we discuss approaches that are related to our new primitives.

Anonymous Credentials (AC) [PS16, San20, HS21, CLPK22] are a valuable tool for strong authentication with built-in access control and play a vital role in privacy safeguarding. In an AC system, credentials are issued by trusted issuer(s) and certify some set of user attributes. Credential holders can show that they possess a credential whose hidden attributes satisfy some predicate in a zero-knowledge manner. It is crucial to maintain the unlinkability

between multiple showings of the same credential, even in situations where issuers and verifiers collude. AC technologies have been applied in diverse domains, including PrivacyPass by Cloudflare [DGS⁺18], Google’s enhanced variant integrated into the Trust Tokens API [KLOR20], and the DIT proposal by Facebook [HIJ⁺21]. The Hyperledger system [C⁺16] facilitates the integration of anonymous credentials with blockchains by supporting CL credentials [CL04] through a trusted third-party issuer. However, such a system becomes vulnerable in situations where the issuer’s behavior turns malicious. There are various proposals for threshold-issuance AC schemes [SABB⁺18, MSM23, DKL⁺23]. Said systems leverage threshold cryptography and enable multiple authorities to jointly issue credentials, hence proving well-suited for distributed ledger systems. Another line of research in AC systems is focused on attaining the multi-authority feature. This feature facilitates a compact and efficient showing of multiple credentials issued by various issuers [HP23, MBG⁺23]. zkLogin [BCJ⁺24], functioning similarly to AC systems, allows users to demonstrate specific attributes derived from traditional web services, enabling privacy-preserving authentication for cryptocurrency transactions. By utilizing providers in the OpenID Connect network, such as Google and Facebook, zkLogin establishes a link between real-world identities and cryptographic keys, thus removing the burden of key management for digital wallets. However, a notable limitation of these systems is their inability to address the critical requirement of Sybil-resistant credential issuance, a property that holds significant importance in numerous applications.

Anonymous Counting Tokens (ACT) is a new primitive introduced recently in a work by Benhamouda et al. [BRS23]. This novel primitive empowers users to acquire tokens on private messages of their preference, while enabling issuers or verifiers to enforce rate limits on the number of tokens that users can obtain or redeem for each message. An intriguing challenge that arises in the design phase of an ACT system is the apparent conflict between the desire for anonymity, which prevents mapping tokens to user identities, and the need to set limits on individual user contributions. Additionally, it is important to ensure that diverse users can acquire anonymous tokens for the same message, while keeping this information concealed from the issuer to preserve the property of unlinkability. Existing tools in the field of secure computation including Multi-Party Computation (MPC) [Gol09] and secure aggregation constructions [BIK⁺17, BBG⁺20], address the problem of computing aggregates, while preserving privacy for individual contributions. However, these solutions fall short in providing rate limiting for user inputs, particularly in scenarios where user contributions must be generated while maintaining user anonymity. The work in [BRS23] proposes two approaches for the construction of ACTs. The first approach employs PRF evaluation as the token issuance mechanism, incorporating keyed verification that is only advantageous in scenarios where the issuer and verifier are the same entity. The second approach harnesses the concept of Equivalence Class Signatures (EQS) [FG18, FHS19] to establish an ACT scheme. However, it is worth noting that both approaches operate within a centralized setting, where a single party, known as the issuer, is responsible for issuing tokens to users. In a

concurrent study [KRB⁺24], the authors propose anonymous tokens with decentralized issuance based on blind multi-signatures. However, their approach utilizes a multi-issuer model instead of a threshold-based issuance and does not support attributes. Moreover, they enforce rate limiting or single-use property during issuance, relying on the assumption that the issuer knows the user’s identity at the time of issuance. As a result, it fails to ensure user anonymity against the issuer.

B Additional Preliminaries

B.1 Notation

Let $\lambda \in \mathbb{N}$ represent the security parameter. For any finite set S , we use the notation $x \leftarrow S$ or $x \leftarrow^* S$ to indicate that x is uniformly sampled from S . For an algorithm A , we denote the process of running A on input x and utilizing uniformly random coins r as $y \leftarrow A(x; r)$, where the result is assigned to y . In this context, $[i, j]$ represents a set comprising all integers within the range from i to j , inclusive. We assume all algorithms are probabilistic polynomial-time (PPT) unless explicitly stated otherwise. Public parameters are assumed as an implicit input to all algorithms in a scheme. negl is a negligible function in the security parameter λ , that is for any positive integer k and for any large enough λ , $\text{negl}(\lambda) \leq 1/\lambda^k$.

B.2 Pedersen Commitments

We adopt the Pedersen commitment as our commitment scheme with binding and hiding properties. We utilize Pedersen commitment over a group $\mathbb{G}$ of large order p in which the discrete logarithm assumption holds. The scheme is defined as a tuple of the following algorithms:

- $\text{pp} \leftarrow \text{Ped.Setup}(1^\lambda)$: This algorithm inputs security parameter λ and outputs public commitment parameters pp as two generators g, u of the group $\mathbb{G}$.
- $\text{cm} \leftarrow \text{Ped.Commit}(m; r)$: This algorithm generates a commitment cm of message m using randomness r . The cm is of the form $\text{cm} = g^r \cdot u^m$ where $r \leftarrow^* \mathbb{F}_p$.
- $\{0, 1\} \leftarrow \text{Ped.Open}(\text{cm}, r, m)$: To open a commitment cm , the sender sends $\{r, m\}$ to the receiver. The receiver verifies the opening by checking whether $\text{cm} = g^r \cdot u^m$ and outputs a bit $b \in \{0, 1\}$.

The binding property of the commitment scheme necessitates that the prover lacks knowledge of the discrete log relation between generators g, u .

Generalized Pedersen Commitment. There is an important generalization of the Pedersen commitment scheme [Ped91], with which a prover can commit to multiple messages $(m_0, \dots, m_{n-1})$ for $n \in \mathbb{Z}^+$. The generalized Pedersen commitment over a group G of prime order p is defined as a tuple of the following algorithms:

- $\text{pp} \leftarrow \text{Ped.Setup}(1^\lambda, n)$: This algorithm inputs security parameter λ and outputs public commitment parameters pp consisting of $n + 1$ elements of the group $\mathbb{G}$ denoted as $g, u_0, u_1, \dots, u_{n-1}$.
- $\text{cm} \leftarrow \text{Ped.Commit}(m_0, \dots, m_{n-1}; r)$: To commit to messages $(m_0, \dots, m_{n-1})$, where $m_i \in \mathbb{F}_p$ for $i \in [0, n-1]$, the algorithm samples a random value $r \xleftarrow{\$} \mathbb{F}_p$, computes $\text{cm} = g^r \cdot \prod_{i=0}^{n-1} u_i^{m_i}$ and outputs cm .
- $\{0, 1\} \leftarrow \text{Ped.Open}(\text{cm}, r, m_0, \dots, m_{n-1})$: The algorithm inputs opening of commitment cm as $\{r, m_0, \dots, m_{n-1}\}$ and verifies the opening by checking whether $\text{cm} = g^r \cdot \prod_{i=0}^{n-1} u_i^{m_i}$. It outputs a bit $b \in \{0, 1\}$.

B.3 Zero-Knowledge Proofs of Knowledge

We define zero-knowledge proofs of knowledge (ZKPoK) and discuss their non-interactive variants (NIZK). In this paper, we require protocols to prove knowledge of discrete logarithm relations. This can be efficiently realized by utilizing Sigma protocols [CDS94, Cra96], which are efficient instantiations of ZKPoK and can be made non-interactive using the Fiat-Shamir heuristic [FS86]. Moreover, we need to prove a pairing product equation and thus also rely on the Groth-Sahai NIZK proof system [GS12]. Let $\mathcal{R} : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}$ be a NP-witness-relation with corresponding NP-language $\mathcal{L} := \{x : \exists w \text{ s.t. } \mathcal{R}(x, w) = 1\}$. A non-interactive zero-knowledge proof (NIZK) system for $\mathcal{R}$ has the following algorithms:

- $\text{crs} \leftarrow \text{ZKSetup}(1^\lambda, \mathcal{R})$: Generates common reference string crs for the prove relationship $\mathcal{R}$.
- $\pi \leftarrow \text{ZKPoK}(\mathcal{R}, \text{crs}, x, w)$: Generates a proof π that the prover knows a witness w such that the input statement x satisfies the relation $\mathcal{R}(x, w)$.
- $\{0, 1\} \leftarrow \text{ZKVerify}(\text{crs}, x, \pi)$: Efficiently verifies the correctness of the proof for a statement x . Outputs a bit $b \in \{0, 1\}$, where $b = 1$ shows that proof π is accepted, and $b = 0$ otherwise.

The notation used to represent ZK proofs is $\text{ZKPoK}\{w : \text{statement about } w\}$, as introduced by Camenisch *et al.* [CS97]. This notation indicates showing in zero-knowledge that certain secret values w fulfill the statement listed after the colon. A ZKPoK system must satisfy the following properties: (I) Zero-knowledge: The verifier $\mathcal{V}$ gains no additional knowledge beyond the validity of statements during the protocol execution, (II) Simulation sound: It is computationally infeasible for any prover $\mathcal{P}$ to persuade the verifier $\mathcal{V}$ of the validity of an invalid statement (selected by $\mathcal{P}$), even after having seen a polynomial number of simulated proofs for their chosen statements.

B.4 Oracle Systems

Privacy-Preserving Oracles [ZCC⁺16, ZMM⁺20], leverage cryptographic techniques that enable a user ($\mathcal{P}$) to prove to a verifier ($\mathcal{V}$) arbitrary statements about

their data, which is securely retrieved from an authoritative web server ($\mathcal{S}$). This process ensures data privacy is preserved throughout. For instance, $\mathcal{P}$ can provide an oracle proof $\pi^{\mathcal{O}}$ to convince $\mathcal{V}$ that she is over 18, based on information sourced from her online U.S. State Department account ($\mathcal{S}$) without revealing any additional data beyond this statement. Oracles play a pivotal role in granting a wide range of applications— both in blockchain and traditional systems— access to private and public web data. Notable examples include credential creation from legacy data in DID systems and feeding real-world data to privacy-preserving smart contracts in Decentralized Finance (DeFi) [W3C18, BCC⁺21].

C Proofs

C.1 Security Analysis of tSW

Theorem 1. *If k -OMDL and CDH assumptions hold, then, the tSW scheme in Section 2.2 is statically TS-UF-1 secure in the AGM+ROM.*

As shown by Bach and Loss in [BL22], the threshold BLS signature scheme can be shown adaptively TS-UF-1 secure [BCK⁺22], i.e., under a strong notion where the adversary is also allowed to query partial signatures for the forged message with up to $\frac{N}{2}$ corruptions, under the k -OMDL assumption in the ROM and AGM. As we discussed in Section 2, for our purpose static security suffices. But every scheme with adaptive corruptions is clearly also secure under static ones.

There are two possible cases: 1) The adversary outputs a forgery for a new pair (i^*, m^*) , where i^* has not been queried before. In this case, we can reduce to tBLS security, or 2) The adversary outputs a forgery for a new pair (i^*, m^*) , where there exists a previously queried pair (i, m) , such that $i^* = i$ and $m^* \neq m$, then we can construct an algebraic adversary that can solve the computational Diffie-Hellman (CDH) problem with a non-negligible advantage. We discuss these two cases in the following claims.

Claim. Under Case 1, the tSW scheme is statically TS-UF-1 secure under the same assumptions as in [BL22] (Theorem 4.1).

Proof (Proof sketch). We can turn any adversary against tSW into one against threshold BLS (tBLS). Note that the reduction can set up $u = g^v$ and pk as well as $\{pk_i\}_{i \in N}$ are available to the reduction. For every partial signing query for message m of the tSW adversary, we can forward m to the partial signing query of the tBLS challenger. For result σ_i we compute $\sigma_i \cdot pk_i^{vm}$ and give it to the tSW adversary. It can be seen that this is a valid partial signature for tSW. For the forgery attempt σ^* , the reduction outputs $\sigma^*(pk^{vm^*})^{-1}$ to the tBLS challenger. It can easily be checked that the simulation is perfect and any valid forgery for tSW is one for tBLS under the same winning conditions.

Claim. Under Case 2, the tSW scheme is statically TS-UF-1 secure under the CDH assumption.

Proof (Proof sketch). Suppose the message (i, m^*) and signature σ^* as the forgery returned by the adversary. We first discuss the reduction for the non-threshold setting. By the correctness of the scheme, we know that (i, m^*, σ^*) satisfies the verification equation, i.e., that $e(\sigma^*, g) = e(\mathbf{H}(i) \cdot u^{m^*}, \mathbf{pk})$. We define $\Delta m = m^* - m$, which must be nonzero. We now show that if there is a non-negligible advantage in the adversary's success probability, we can construct a simulator that solves the CDH problem.

The simulator is given as input values $g, g^\alpha, u \in \mathbb{G}_1$; its goal is to output u^α . The simulator behaves like the EUF-CMA challenger, with the following differences:

- **Setup Phase.** Initialize sets $\mathcal{H} := \{1, \dots, n\}, \mathcal{C} := \emptyset$.
- **Corruption Queries.** Adversary may corrupt a set $\mathcal{C}$ of parties by submitting an index $j \in \mathcal{C}$. The simulator randomly samples the secret key $\mathbf{sk}_j \in \mathbb{F}_p$, returns the secret key of P_j , and sets $\mathcal{H} = \mathcal{H} \setminus \mathcal{C}$. Henceforth, the adversary has full control over parties in set $\mathcal{C}$.
- **Key Generation.** It sets the public key v to g^α received in the challenge. The shares $\mathbf{sk}_{i \in \mathcal{C}}$ are sampled in the Corruption phase and known to the adversary. For $j \in [t-1] \setminus \mathcal{C}$, the simulator randomly samples secret keys $\mathbf{sk}_j \in \mathbb{F}_p$ and then simulates the remaining shares $\mathbf{pk}_j, j \in [t, n]$ consistently using Lagrange interpolation given the available $t-1$ set of shares and v (henceforth, it does not know the corresponding secret key).
- **Signing Queries.** When the adversary submits a pair $j, (i, m)$ for $j \in \mathcal{H}$, the simulator programs the random oracle $\mathbf{H}$. It keeps a list of queries and responses. It answers queries of the form $\mathbf{H}(i)$ in a special way, as follows. It chooses a random $r \xleftarrow{\$} \mathbb{Z}_p$ and programs the random oracle at i as $\mathbf{H}(i) = g^r / u^m$. Now the simulator can compute σ_j , since we have $\mathbf{H}(i) \cdot u^m = g^r / u^m \cdot u^m$. Therefore, the simulator computes $\sigma_j = (\mathbf{H}(i) \cdot u^m)^{\mathbf{sk}_j} = (\mathbf{pk}_j)^r$.
- **Output Determination.** The simulator continues interacting with the adversary until the adversary succeeds in responding with a signature σ^* on message (i, m^*) . Let $\mathcal{S} \subset \{1, \dots, n\}$ denote the subset of parties for which the adversary made a signing query of the form $j, (i, m^*)$. The simulator checks whether $|\mathcal{C} \cup \mathcal{S}| \leq t$ and $e(\sigma^*, g) = e(\mathbf{H}(i) \cdot u^{m^*}, v)$. Next, the simulator calculates $t-1$ partial signatures $\{\sigma_c\}_{c \in C}$, where $C = \{c_1, \dots, c_{t-1}\}$ represents the parties for which the simulator knows the corresponding secret keys. The simulator then computes $\sigma_{c_t}^* = (\frac{\sigma^*}{\prod_{c_j \in C} \sigma_{c_j}^{\ell_j(0)}})^{\ell_t(0)^{-1}}$, with ℓ_j being the j^{th} Lagrange polynomial for points $c_1, \dots, c_t$. Now, dividing the verification equation for the signature $\sigma_{c_t}^*$ by the verification equation for the signature σ_{c_t} on (i, m) , it obtains: $e(\sigma_{c_t}^* / \sigma_{c_t}, g) = e(u^{\Delta m}, \mathbf{pk}_{c_t})$. Noting that $\mathbf{pk}_{c_t} = g^{(\alpha - \sum_{c_j \in C} \ell_j(0) \mathbf{sk}_{c_j}) \ell_t(0)^{-1}}$, one can successfully derive a solution to the CDH problem, $u^\alpha = (\sigma^* \cdot \sigma^{-1})^{\frac{\ell_t(0)}{\Delta m}} \cdot u^{\sum_{c_j \in C} \ell_j(0) \mathbf{sk}_{c_j}}$.

C.2 Security Analysis of tACT

In this section we present the security proofs of our tACT construction.

Theorem 2. [Unforgeability] *If tSW provides unforgeability, the cut-and-choose scheme is sound, and the ZKPoK argument is knowledge sound, then, the tACT scheme in Algorithm 1 satisfies unforgeability from Definition 3.*

Proof. Assume there is an adversary $\mathcal{A}$ that wins the unforgeability security game in Figure 4 with non-negligible probability. We construct three PPT adversaries: $\mathcal{B}_{\text{tSW}}$ which breaks the unforgeability of the tSW, $\mathcal{B}_{\text{SND}}$ which breaks the soundness of the cut-and-choose scheme, and $\mathcal{B}_{\text{zkSND}}$ breaks the soundness of the zero-knowledge argument. We show that:

$$\begin{aligned} \text{Adv}_{\text{tACT}, \mathcal{A}}^{\text{uf}} &\leq (T + L + 1) \cdot \text{Adv}_{\text{tSW}, \mathcal{B}_{\text{tSW}}}^{\text{uf}}(\lambda) + T \cdot \text{Adv}_{\text{SND}, \mathcal{B}_{\text{SND}}}^{\text{sound}}(\lambda) \\ &\quad + T \cdot \text{Adv}_{\text{SND}, \mathcal{B}_{\text{zkSND}}}^{\text{zk sound}}(\lambda). \end{aligned} \quad (1)$$

Construction of the adversaries. Let us first construct $\mathcal{B}_{\text{tSW}}$. $\mathcal{B}_{\text{tSW}}$ obtains pk from the challenges for the tSW scheme and provides it as public parameters to $\mathcal{A}$. $\mathcal{B}_{\text{tSW}}$ answers signing queries $\pi^* = \{\text{cm}, \{\text{cm}_k\}_{k \in [1, n]}, \text{cm}_k\}$ from $\mathcal{A}$ as follows. If π^* fails to verify, abort. Else, $\mathcal{A}$ queries the tSW challenger for signatures and returns the result. We consider the output $\mathbf{h} := (\text{token}_i := s_i, \pi_{\text{tok}_i})_{i \in [T+1]}$ that $\mathcal{A}$ generates. We write $\pi_{\text{tok}, i} = (\{s'_{k,i}\}_{k \in [1, n]}, C_i, \{R'_{k,i}\}_{k \in C_i}, \text{pk}', \pi_{\text{zk}})$. At the end, $\mathcal{B}_{\text{tSW}}$ does the following:

- If $\mathcal{A}$ produced a Type-1 forgery, it uniformly at random chooses $i^* \in [T + 1]$ and return the i^* th token $\pi_{\text{tok}_{i^*}} = (\{s'_{k,i^*}\}_{k \in [1, n]}, C_{i^*}, \{R'_{k,i^*}\}_{k \in C_{i^*}}, \text{pk}', \pi_{\text{zk}})$ to the tSW challenger as its forgery.
- If $\mathcal{A}$ produced a Type-2 forgery with set $\tau \subset [T]$. Assume without loss of generality that $|\tau| = L + 1$. $\mathcal{B}_{\text{tSW}}$ select uniformly at random $i^* \in \tau$ and outputs the i^* -th token $\pi_{\text{tok}_{i^*}} = (\{s'_{k,i^*}\}_{k \in [1, n]}, C_{i^*}, \{R'_{k,i^*}\}_{k \in C_{i^*}}, \text{pk}', \pi_{\text{zk}})$ to the tSW challenger as its forgery.

Now, let us construct $\mathcal{B}_{\text{SND}}$. $\mathcal{B}_{\text{SND}}$ runs the cut-and-choose challenger and does the following:

- If $\mathcal{A}$ produced a Type-1 forgery, it outputs $\perp$ to the cut-and-choose soundness challenger.
- If $\mathcal{A}$ produced a Type-2 forgery with set $\tau \subset [T]$. Assume without loss of generality that $|\tau| = L + 1$. $\mathcal{B}_{\text{SND}}$ select uniformly at random $j^* \in [T]$ and outputs the j^* -th query $\pi^* = (\text{cm}_{j^*}, \{\text{cm}_{k,j^*}\}_{k \in [1, n]}, \text{cm}_k)$ made by $\mathcal{A}$ to tACT.tlssue to the cut-and-choose challenger (as a proof of an invalid statement).

Now, we construct $\mathcal{B}_{\text{zkSND}}$. $\mathcal{B}_{\text{zkSND}}$ runs the ZK soundness challenger and does the following:

- If $\mathcal{A}$ produced a Type-1 forgery, it outputs $\perp$ to the ZK soundness challenger.
- If $\mathcal{A}$ produced a Type-2 forgery with set $\tau \subset [T]$. Assume without loss of generality that $|\tau| = L+1$. $\mathcal{B}_{\text{zkSND}}$ select uniformly at random $j^* \in [T]$ and outputs the j^* -th query $\pi_{\text{token}}^* = \{\{s'_{k,j^*}\}_{k \in [1,n]}, C_{j^*}, \{R'_{k,j^*}\}_{k \in C}, \pi_{\text{zk},j^*}, \text{cm}_{j^*}, \text{cm}_{k,j^*}, \text{pk}'_{j^*}\}$ made by $\mathcal{A}$ to tACT.Verify to the ZK soundness challenger (as a proof of an invalid statement).

Remark that $\mathcal{B}_{\text{tSW}}$, $\mathcal{B}_{\text{SND}}$, and $\mathcal{B}_{\text{zkSND}}$ cannot know (in polynomial time) whether the output to their tSW/cut-and-choose/ZK challenger is valid, i.e., if they will win the $\text{tSW}_{\mathcal{B}_{\text{tSW}}}(\lambda)$, $\text{SND}_{\mathcal{B}_{\text{SND}}}(\lambda)$, and $\text{zkSND}_{\mathcal{B}_{\text{zkSND}}}(\lambda)$ games. However, this is not an issue to prove that the advantage of at least one of those two adversaries is non-negligible if the advantage of $\mathcal{A}$ in unforgability experiment is non-negligible. Finally, in order to extract r_i and a_i (following the experiment of unforgability of tACT [section 2.1](#)), we run the extractor Ext of the underlying ZKPoK's extractor to extract the committed values r_i and a_i .

Our proof strategy is the following. We define events $\mathsf{E}_{1\text{tSW},i}$, $\mathsf{E}_{2\text{tSW},k}$, $\mathsf{E}_{2,\text{SND},j}$, and $\mathsf{E}_{2,\text{zkSND},j}$ for $i \in [T+1]$, $k \in [L]$, $j \in [T]$ so that:

- Exactly one of these events must happen if the adversary wins.
- Conditioned on any of these events, either $\mathcal{B}_{\text{tSW}}$ or $\mathcal{B}_{\text{SND}}$, or $\mathcal{B}_{\text{zkSND}}$ wins EUF-CMA or soundness respectively.

Note that there is no need for these events to be efficiently checked for the proof to go through.

Definition of Events. Let us first define events $\mathsf{E}_{1\text{tSW},i}$ that are associated with a Type-1 forgery. In that case, $\mathcal{A}$ generated $T+1$ tokens by doing T signature queries. $\mathcal{A}$ queries Sign only T times and there are $T+1$ valid tokens $\{\pi_{\text{tok}_i}\}_{i \in [T+1]}$, there exists $i \in [T+1]$ so that π_{tok_i} (and Token_i) is not in the query's list queried to Sign . And π_{tok_i} is a forgery for the tSW scheme.

We define $\mathsf{E}_{1\text{tSW},i}$ the event that $\mathcal{A}$ produced a Type-1 forgery and π_{tok_i} (and Token_i) is not in the query's list queried to Sign , and no event $\mathsf{E}_{1\text{tSW},i'}$ happened for $i' < i$. (This last constraint is to ensure that the events $\mathsf{E}_{1\text{tSW},i}$ are disjoint.) Equivalently, $\mathsf{E}_{1\text{tSW},i}$ is defined as the event that $\mathcal{A}$ produced a Type-1 forgery and π_{tok_i} if the $\pi_{\text{tok}_{i'}}$ that is not in the query's list queried to Sign .

Let us now consider Type-2 forgeries. Let us show that at least one of the following events must happen:

- Event $\mathsf{E}_{2\text{tSW},k}$: $\mathcal{A}$ made a Type-2 forgery (and not a Type-1 forgery) for a set $\tau = i_1, \dots, i_L$ with $i_1 < \dots < i_L$, so that $\pi_{\text{tok}_{i_k}}$ is not in the query's list queried to Sign , and so that no event $\mathsf{E}_{2\text{tSW},k'}$ happened for $k' \leq k$.
- Event $\mathsf{E}_{2,\text{SND},i}$:
 - $\mathcal{A}$ made a Type-2 forgery (and not a Type-1 forgery) and,
 - its i -th signing query is $(\text{Token}_i, \pi_{\text{tok}_i})$ where π_{tok_i} is valid but proves a wrong cut-and-choose statement on the Token_i , and,
 - neither event $\mathsf{E}_{2\text{tSW},k}$ nor $\mathsf{E}_{2,\text{SND},j}$ happened for $j' < j$ and any k .

The event $E_{2,\text{zkSND},i}$ is similar to $E_{2,\text{SND},i}$ but there π_{tok_i} is valid but proves a wrong ZK statement on the Token_i , and, non of the events $E_{2\text{tSW},k}$ or $E_{2,\text{SND},j}$ or $E_{2,\text{zkSND},j}$ happened for $j' < j$ and any k .

Let us assume by contradiction that none of the events happened (assuming $\mathcal{A}$ made a Type-2 forgery). Then since no $E_{2\text{tSW},k}$ happened, each $\pi_{tok_{i_k}}$ is in the query's list queried to Sign. Since there are $L + 1$ of them and there are only L clients, at least two of them were made for the same client. These two queries are for the same message on which the Type-2 forgery was made. But since the cut-and-choose proofs were all proving valid statements (as no event $E_{2,\text{SND},i}$ occurred), this is a contradiction. So at least one of the events must happen.

Conclusion of the Proof. Since at least one of the events $E_{1\text{tSW},i}$, $E_{2\text{tSW},k}$, and $E_{2,\text{SND},i}$ must happen and these events are disjoint, we have that from the total probability rule:

$$\begin{aligned} \text{Adv}_{\text{tACT},\mathcal{A}}^{\text{uf}} &= \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1] = \sum_{i=1}^{T+1} \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{1\text{tSW},i}] \\ &+ \sum_{k=1}^L \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{2\text{tSW},k}] + \sum_{j=1}^T \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{2,\text{SND},j}] \end{aligned}$$

In addition, looking at the definition of $\mathcal{B}_{\text{tSW}}$ and from the fact this reduction perfectly simulates the oracles for $\mathcal{A}$:

$$\begin{aligned} &\sum_{i=1}^{T+1} \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{1\text{tSW},i}] \\ &= \sum_{i=1}^{T+1} \Pr[\text{uf}_{\mathcal{B}_{\text{tSW}}}(\lambda) = 1 \wedge \text{Event}_{1\text{tSW},i} | i^* = i] \\ &= \Pr[i^* = i]^{-1} \sum_{i=1}^{T+1} \Pr[\text{uf}_{\mathcal{B}_{\text{tSW}}}(\lambda) = 1 \wedge E_{1\text{tSW},i} \wedge i^* = i] \\ &\leq \Pr[i^* = i]^{-1} \sum_{i=1}^{T+1} \Pr[\text{uf}_{\mathcal{B}_{\text{tSW}}}(\lambda) = 1 \wedge i^* = i] \\ &= (T + 1) \cdot \Pr[\text{uf}_{\mathcal{B}_{\text{tSW}}}(\lambda) = 1]. \end{aligned}$$

Similarly, we have,

$$\begin{aligned} &\sum_{k=1}^L \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{2\text{tSW},k}] \leq L \cdot \Pr[\text{uf}_{\mathcal{B}_{\text{tSW}}}(\lambda) = 1], \\ &\sum_{j=1}^T \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge E_{2,\text{SND},j}] \leq T \cdot \Pr[\text{SND}_{\mathcal{B}_{\text{SND}}}(\lambda) = 1]. \end{aligned}$$

$$\sum_{j=1}^T \Pr[\text{uf}_{\mathcal{A}}(\lambda) = 1 \wedge \mathbf{E}_{2\text{zkSND},j}] \leq T \cdot \Pr[\text{zkSND}_{\mathcal{B}_{\text{zkSND}}}(\lambda) = 1].$$

Together this concludes Equation (1).

Theorem 3. *[Unlinkability] If the Ped.Commit scheme provides hiding property, the ZKPoK scheme provides zero-knowledge property, and H is modeled as a random oracle, then the tACT scheme in Algorithm 1 satisfies unlinkability from Definition 4.*

Proof. We conduct the proof using hybrid games:

Game₀: This game represents the unlinkability security game.

Game₁: This game is similar to the previous one, except now all the cut-and-choose proofs are simulated.

Game₀-Game₁: This hybrid is computationally indistinguishable from the previous one due to the hiding property of the cut-and-choose argument.

Game₂: This game is similar to the previous one except that in any Unblind computation, instead of computing $C \leftarrow H'(\text{Token})$, it chooses C uniformly at random.

Game₁-Game₂: This game is indistinguishable from the previous one assuming H' is modeled as a random oracle.

Game₃: This game is similar to the previous one, except now all the ZK proofs are simulated.

Game₂-Game₃: This hybrid is computationally indistinguishable from the previous one due to the zero-knowledge property of the ZK argument.

Game₄: This game is similar to the previous one except that for every token request and associated unblind query, $\mathbf{a}$ is chosen uniformly at random.

Game₃-Game₄: This game is indistinguishable from the previous one assuming Ped.Commit scheme provides the hiding property.

In the final hybrid, the outcome of any token request is unrelated to the inputs $\mathbf{a}$. The result of an unblind request solely comprises the underlying message and remains independent to the output of the corresponding token request. In this last game, no component from the computation of the token request is utilized during unblinding, except for the message itself. Consequently, the adversary's advantage is 0, leading to the conclusion of the proof.

C.3 Security Analysis of S3ID

Now, we provide the security proofs of the S3ID scheme.

Theorem 4. *[Unforgeability] If tSW provides unforgeability security and ZKPoK is sound, then, the S3ID scheme in Algorithm 2 satisfies unforgeability from Definition 6.*

Proof. The proof of unforgeability for S3ID is similar to the proof in [Theorem 2](#). It relies on the soundness property of the ZKPoK and the unforgeability of the tSW scheme. Specifically, the adversary attempting to forge is faced with the following possibilities:

- Computing an invalid signature ζ (while computing a valid τ and its corresponding ZKPoK proof π).
- Computing an invalid τ along with a proof π that passes the verification (while computing a valid signature ζ).

It is evident that the adversary cannot succeed in either case without breaking the unforgeability of the tSW scheme in the first scenario or violating the soundness property of the ZKPoK in the latter case.

Theorem 5. *[Sybil-resistance] If tACT provides unforgeability security and ZKPoK is sound, then, the S3ID scheme in [Algorithm 2](#) satisfies Sybil-resistance from [Definition 7](#).*

Proof. This is a direct result of the unforgeability of the tACT scheme proved in [Theorem 2](#).

Theorem 6. *[Strong Unlinkability] If the Ped.Commit scheme provides the hiding property, H is modeled as a random oracle, ZKPoK provides zero-knowledge property, and the tACT scheme is unlinkable, then the S3ID scheme in [Algorithm 2](#) satisfies strong unlinkability from [Definition 8](#).*

Proof. We proceed with the proof using hybrid games:

Game₀: This game is the unlinkability game in [Figure 8](#).

Game₁: This game is similar to the previous one except now all the ZKPoK proofs are simulated.

Game₀-Game₁: This hybrid is computationally indistinguishable from the previous one due to the zero-knowledge property of the ZKPoK argument.

Game₂: This game is similar to the previous one except that in any AppCred computation, it chooses τ uniformly at random instead of computing $\tau = g^r \cdot u_0^{|\mathbb{Q}|k} \cdot \prod_{i \in \mathbb{Q}} u_i^{a_i}$.

Game₁-Game₂: This game is indistinguishable from the previous one assuming the hiding property of the commitment τ and the zero-knowledge property of the ZKPoK.

Game₃: This game is similar to the previous one except that in any AppCred computation, instead of computing $\text{tg} = \text{PRF}(k, l)$, it chooses tg uniformly at random.

Game₂-Game₃: This game is indistinguishable from the previous one assuming PRF scheme.

Game₄: This game is similar to the previous one except that for every token request and associated unblind query, $\mathbf{a}$ is chosen uniformly at random.

Game₃-Game₄: This game is indistinguishable from the previous one assuming **Ped.Commit** scheme provides the hiding property.

In the last hybrid, token requests become independent of **a**. Unblind request outputs solely include the underlying message and are unrelated to associated token request outputs. Consequently, the adversary's advantage is 0, concluding the proof.